

Propaq: A Python package for Heisenberg Propagation

Hrishikesh Belagali^{1,2} and Ryan LaRose^{1,3,4,2}

¹Department of Computational Mathematics, Science, and Engineering, Michigan State University, East Lansing, MI 48823, USA

²Center for Quantum Computing, Science, and Engineering, Michigan State University, East Lansing, MI 48823, USA

³Department of Electrical and Computer Engineering, Michigan State University, East Lansing, MI 48823, USA

⁴Department of Physics and Astronomy, Michigan State University, East Lansing, MI 48823, USA

We introduce Propaq, a performant and flexible Python package implementing Heisenberg propagation techniques for classically simulating quantum circuits. Propaq is compatible with major quantum software packages like Qiskit and Cirq, and matches or outperforms similar state-of-the-art packages for Heisenberg propagation. Beyond this, Propaq has multiple distinguishing features for simulation and research. These include Heisenberg propagation in any basis, hybrid Schrödinger-Heisenberg simulation, custom truncation and noisy simulation methods, and detailed logging information for analyzing simulation accuracy, runtime, and memory usage. We describe these features, how to use Propaq for research, and its backend implementation.

1 Introduction

Classically simulating quantum circuits is important practically to benchmark algorithms, error mitigation/correction techniques, and other protocols. Classical simulation is also important theoretically to refine the boundaries of quantum advantage [1] and ultimately to understand the power of quantum computation. In this vein, multiple famous theorems about quantum computation have been discovered through the lens of simulating circuits. Two prominent examples are the Gottesman-Knill theorem, which states that an important subclass of quantum operations known as Clifford operations are efficiently simulable [2], and the time evolving block decimation algorithm, which shows that weakly entangled one-dimensional circuits are efficiently

simulable [3]. Additional theorems have been proved in this context, especially recently for efficient classical simulation with noisy quantum circuits [4–7].

Recently, the classical simulation technique underlying the Gottesman-Knill theorem has gained traction due to challenges to quantum advantage claims [8–10]. The simulation technique is commonly known as either sparse Pauli dynamics [9, 10] or Pauli propagation [11]. Subsequently, Majorana propagation — the same underlying technique but working in the Majorana basis instead of the Pauli basis — was introduced [12]. Software packages for these techniques closely followed, with implementations in major quantum software packages like Qiskit and other standalone/plugin packages appearing. Software packages include the PauliPropagation.jl [11] and MajoranaPropagation.jl [13] Julia packages, the PAULI-PROP Qiskit add-on [14], the PYRAULI package [15] which is implemented in C with Python bindings, and most recently the MONO-PROP [12, 16] package which supports both Pauli and Majorana propagation.

In this work, we introduce a Python package for Heisenberg propagation that we call Propaq. Propaq is implemented in Rust for performance and has Python front-ends for usability with major quantum software packages. Notably, direct support for Qiskit circuits is provided. We show in numerical benchmarks that Propaq is among the fastest Heisenberg simulators available. Further, Propaq has several features that distinguish it from current state of the art. Notably, Propaq supports Heisenberg propagation in any basis. The Pauli and Majorana bases are built-in to the library and natively supported, and any other user-defined bases are also supported. Propaq also supports hybrid Schrödinger-Heisenberg simulation. Here, instead of back-propagating ob-

Ryan LaRose: Corresponding author: rmlarose@msu.edu

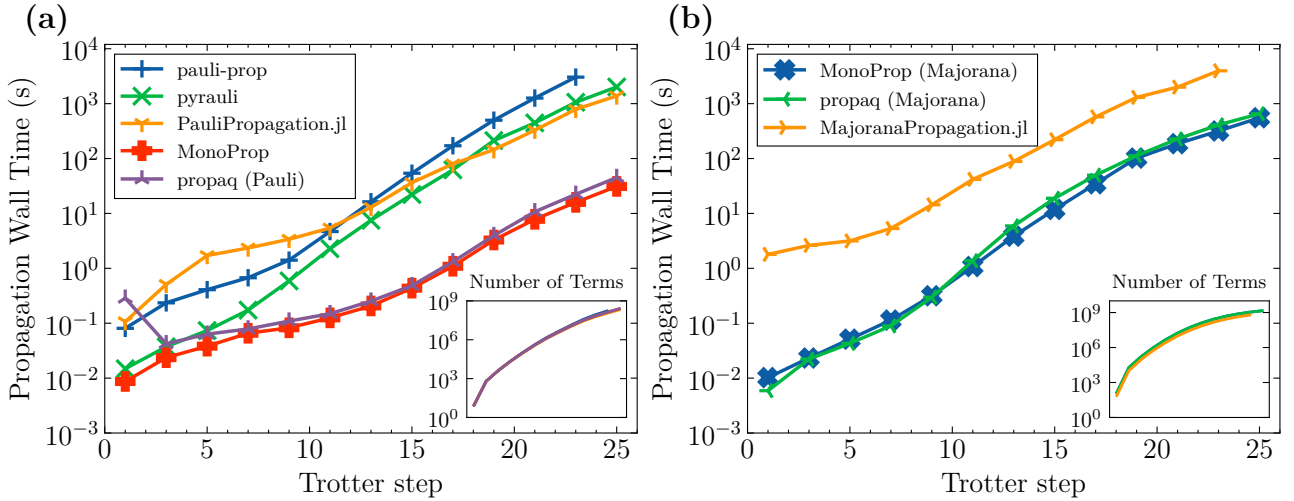


Figure 1: Heisenberg propagation wall-clock runtime and number of terms (inset) for various packages. Here, we benchmark computing expectation values of a Z observable for (a) Trotterized time evolution of a 2D TFIM on a 6×6 square lattice ($J = 1.0$, $h = 0.5$, $\Delta t = 0.1$) and (b) Trotterized time evolution of a 2D Hubbard model on a 3×3 square lattice ($t = 1.0$, $U = 4.0$, $\Delta t = 0.1$). All backends use a coefficient cutoff of 10^{-6} and 64 threads, except for the single-threaded PAULI-PROP package. As shown, Propaq exceeds the performance of all packages in both bases, except for the MONOPROP package which is roughly equal to Propaq.

servables fully through the circuit, we allow forward evolution to a bipartition. In certain cases, this can provide a useful space/time tradeoff and present advantages in simulation, as was explored recently in quantum-classical hybrid simulation [17]. Additionally, Propaq contains logging tools to easily analyze simulation accuracy, providing details on the total number of terms and total L_1 coefficient norm discarded, among other metrics. Propaq also supports custom noise channels and natively supports noisy simulation with any propagation method. For these reasons, we expect Propaq to be a useful package for quantum information researchers and software developers, enabling both fast simulation and future research in (hybrid Schrödinger-)Heisenberg simulation.

To these ends, the rest of the paper is organized as follows. We first provide more details on both the features and performance of Propaq mentioned above, showing benchmarks with other packages and highlighting Propaq’s capabilities, in Sec. 2. We then provide background on Heisenberg propagation in Sec. 3. Section 4 shows how to get started with Propaq and how to use the library through various examples. In Sec. 5, we discuss the design and implementation of Propaq, providing backend implementation and low-level software details. Finally, we provide more performance benchmarks for Propaq in Sec. 6.

2 Features and Performance

2.1 Benchmarks with other packages

In Fig. 1, we evaluate the performance of Propaq against other packages for Heisenberg propagation, including PAULI-PROP, PYRAULI, PauliPropagation.jl, MajoranaPropagation.jl, and MONOPROP. In this benchmark, we show the wall-clock runtime for computing the expectation value of a single Z observable after Trotterized time evolution of a 6×6 transverse-field Ising model (Fig. 1(a)) and a 3×3 Hubbard model (Fig. 1(b)). For all benchmarks, we utilize a coefficient cutoff of 10^{-6} and run on 64 AMD EPYC 7H12 cores. When applicable, we use both Pauli and Majorana propagation. As shown in Fig. 1, Propaq performs similarly to MONOPROP, which both outperform all other packages. For large Trotter steps, where the number of terms reaches a billion, the speedup approaches 100x. While packages have different strengths and performance is somewhat benchmark dependent, we have found this behavior to be generally true for all packages. It is worth noting that PAULI-PROP has a single-threaded implementation while other packages are multi-threaded. In this benchmark, all packages maintain a similar number of terms in the back-propagated operator, and the multi-threaded packages consequently have similar peak memory usage. Memory usage is shown in

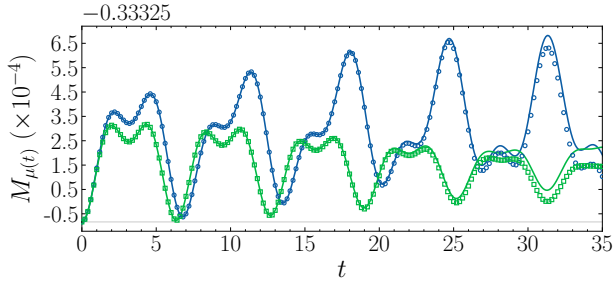


Figure 2: An illustrative example of Propaq’s ability to perform Heisenberg evolution in any basis, demonstrated via time evolution of the three-state (qutrit) Potts model [18]. We subclass Propaq’s core propagation engine to do Heisenberg evolution with Weyl-Heisenberg operators, a higher-dimensional generalization of qubit Pauli operators. Here we compute the magnetizations $M_2(t)$ (blue) and $M_3(t)$ (green) for a quench of the three-state Potts spin chain in the positive oblique regime ($h_2 = 0.1$, $n = 7$) using exact diagonalization (solid) and Weyl-Heisenberg propagation (markers) in Propaq. After creating a subclass to enable propagation in a new basis, all features of Propaq can be used. To illustrate this, we perform backpropagation with a weight cutoff of $w = 4$. As can be seen, results are in excellent agreement with exact diagonalization up to $t \leq 25$ despite the weight cutoff, then start to accumulate error after. When computing these observables, the system is initialized in the fully polarized state.

more detail in Fig. 10 in Sec. 6, which contains additional performance benchmarks.

2.2 Arbitrary basis backpropagation

Although Heisenberg propagation is generally carried out in the Pauli or Majorana basis, it applies to any operator basis. Propaq supports this natively in its design. To implement backpropagation in any basis, the following methods need to be implemented:

1. Products of basis elements,
2. If a term branches under a circuit operation,
3. The operator weight (for truncation support),
4. Diagonal expectation with a basis state,
5. Express gates in terms of basis generators.
6. Hashing, equality and byte representations for term I/O

These are defined by subclassing Propaq’s abstract classes for propagation and implementing

the required abstract methods. Example code is provided in Sec. 4. After this, observable backpropagation can be used as with this custom basis in the same way as with the built-in bases (Pauli and Majorana).

As an example, we use Propaq to compute magnetizations in the Potts model in Fig. 2. Here, we use backpropagation with Weyl-Heisenberg operators, a generalization of Pauli operators to qudits. Creating a subclass for Weyl-Heisenberg propagation allows us to use Propaq for simulating circuits with all features, notably term truncation. In Fig. 2, the magnetizations of the seven-site, three-state Potts spin chain are computed with a weight cutoff of $w = 4$. As shown, the computed expectation values are in excellent agreement with exact diagonalization for short times, up to $t \leq 25$. Afterwards, the expectation values accumulate error due to the $w = 4$ truncation. The complete code in Propaq to implement this example is provided in Appendix A. To the best of our knowledge, Propaq is the only library to support backpropagation in any basis, and this example would not be possible with other packages.

2.3 Hybrid Schrödinger-Heisenberg simulation

In addition to traditional Heisenberg propagation, Propaq supports a hybrid Schrödinger-Heisenberg simulation algorithm. In this approach, a circuit is divided into two parts: the first part is simulated in the Schrödinger picture, and the latter is simulated in the Heisenberg picture. In other words, for circuit $U = U_N \cdots U_1$, this method evaluates expectation values via

$$\text{Tr} [U \rho U^\dagger O] = \text{Tr} [\tilde{\rho}_k \tilde{O}_k], \quad (1)$$

where

$$\tilde{\rho}_k := U_k \cdots U_1 \rho U_1^\dagger \cdots U_k^\dagger \quad (2)$$

and

$$\tilde{O}_k := U_{k+1}^\dagger \cdots U_N^\dagger O U_N \cdots U_{k+1}. \quad (3)$$

This is shown schematically in Fig. 3. The expectation value (1) is computed via a series of overlap computations between $\tilde{\rho}_k$ and the basis-expanded $\tilde{O}_k$. Currently, Propaq supports matrix product states for the forward Schrödinger evolution, but in principle any method is supported. Indeed, the user only needs to define the method for

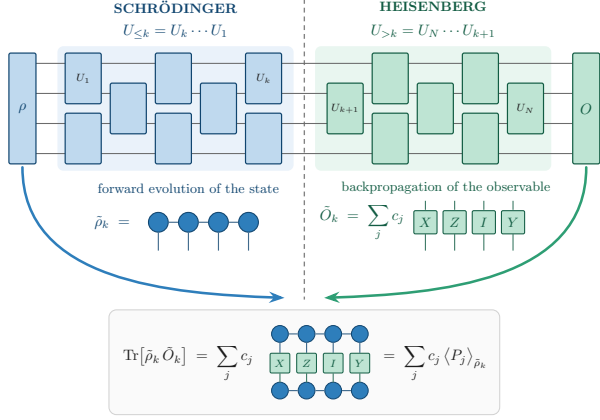


Figure 3: Graphical illustration of the Schrödinger-Heisenberg simulation method (1). The circuit is partitioned at index k (dashed line). Operations before the partition are applied to the initial state ρ via Schrödinger simulation, for example a matrix product state (MPS). Operations after the partition are applied to the observable O via Heisenberg simulation, for example in the Pauli basis as illustrated. Finally, the expectation value is computed as a series of overlaps between the forward-evolved state and the backpropagated observable. In Propaq, forward MPS is currently implemented, but custom methods are support — all that needs to be defined is how to compute overlaps between the forward-evolved state and the backpropagated observable basis elements.

forward simulation (which is commonly done via another package, e.g. Stim for Clifford simulation [19]) and the overlap calculation for the particular basis. We expect this hybrid algorithm to present a useful space-time tradeoff for particular problems and/or compute platforms, and to our knowledge Propaq is the only Heisenberg simulation package to support this. We note that this hybrid algorithm is also compatible with forward evolution on a quantum computer as in [17]. Code for using this hybrid algorithm method is shown in detail in Sec. 4.

2.4 Truncation, logging, and accuracy

A cornerstone of Heisenberg simulators is truncation, used as a heuristic to circumvent the general exponential complexity of the simulation. While this is commonly employed, it is not well understood how to choose the best truncation strategy for a given problem, or what the observable accuracy will be. To address these issues, Propaq enables the use of both commonly used truncation strategies — coefficient and weight truncation — plus custom truncation strategies.

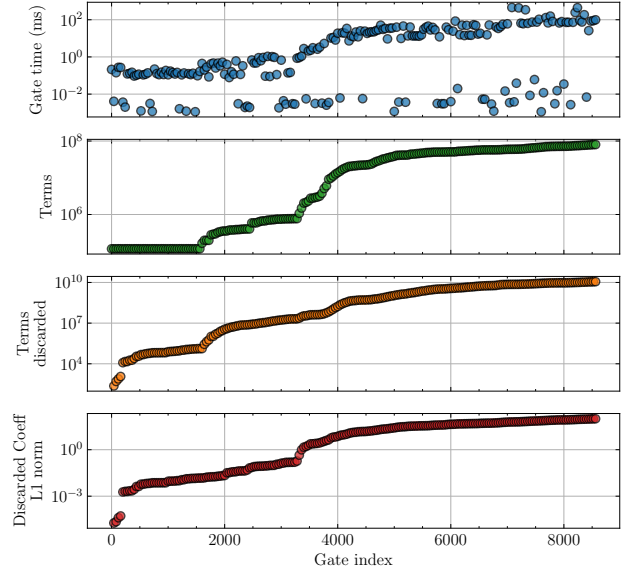


Figure 4: Illustration of some logging/analysis features in Propaq. From top to bottom, this plot shows the time to apply each gate in milliseconds, the cumulative number of terms in the simulation, the cumulative number of terms discarded, and the cumulative L_1 norm of discarded coefficients. The last metric provides an upper bound on observable accuracy. Propaq contains additional logging/analysis metrics not shown here, for example truncation events, total elapsed time, and maximum discarded coefficient L_1 norm. The data shown is for the simulation of a two-layer unitary cluster Jastrow ansatz [20] with Majorana propagation and a coefficient cutoff of 10^{-6} .

Truncation is handled in Propaq via a sequence of truncation strategies, as below.

Listing 2.1. Truncation policies in Propaq.

```
1 from propaq.truncation import WeightTruncator,
   ↪ CoefficientTruncator, TermBudget
2
3 truncation = [
4     CoefficientTruncator(1e-6),
5     WeightTruncator(5),
6     TermBudget(min_terms=1_000)
7 ]
```

This truncation strategy says to discard terms with coefficient magnitude smaller than 10^{-6} , discard terms with weight larger than five, and never drop below one thousand terms. Any combination of these policies can be used in simulation, and custom truncation policies can be defined. Truncation policies are incorporated into propagators at instantiation.

Listing 2.2. Syntax for propagation with truncation.

```
1 from propaq import PauliPropagator
2
3 prop = PauliPropagator(truncation=truncation)
```

The use of propagators to simulate circuits is done by calling the `propagate` or `expectation_value` methods, shown in Sec. 4. We remark that custom propagators in other bases (Sec. 2.2) support truncation in the same manner.

On top of truncation, Propaq contains detailed logging tools to analyze runtime and memory complexity as well as simulation accuracy. First, progress through a simulation can be tracked during runtime with a progress bar.

Listing 2.3. Propaq implements a progress bar to show simulation progress and estimate total runtime.

```
1 from propaq import PauliPropagator
2
3 propagator = PauliPropagator(progress_bar=True)
4 result = propagator.expectation_value(...)
5 # Propagating: 100% 44/44 [00:00<00:00,
   ↪ 273.21gate/s, terms=50]
```

As shown, progress bars show the percentage of gates applied in the circuit, gate application time, and total number of terms in the simulation. The returned `result` contains the number of terms in backpropagated observable after every gate applied in simulation, which is useful for analyzing term growth and simulation complexity.

Even more detailed analyses of the simulation can be enabled with Propaq’s logging tools. One

can log simulation statistics by creating a propagator with a logger object as below.

Listing 2.4. Logging simulation statistics in Propaq.

```
1 from propaq import CoefficientTruncator,
   ↪ Logger, MajoranaPropagator
2
3 logger = Logger("log.jsonl", log_every=1)
4 prop = MajoranaPropagator(
5     truncation=CoefficientTruncator(1e-6),
6     progress_bar=True,
7     logger=logger,
8 )
```

This provides the propagator with logging tools to track timing of various stages of the propagation, as well as truncation statistics, including the L_1 norm of the discarded terms, number of terms discarded, largest magnitude of discarded terms, etc. An example of this functionality is shown in Fig. 4 for simulating a chain of twenty Hydrogen atoms with Pauli propagation and a cutoff of 10^{-6} . Here, we see that, even though nearly 10^8 terms are discarded due to truncation, the discarded coefficient L_1 norm is more mild, between 10^0 and 10^1 , providing a bound on observable accuracy. In addition to providing insight to simulation performance and accuracy, these tools are provided to aid in fine-tuning truncation parameters, and to identify bottlenecks in custom propagation engines.

Finally, we note that Propaq provides API methods to save backpropagated operators to disk in a compressed format. Operators can be loaded from disk into memory all at once, or in a streaming fashion, where terms are loaded one at a time. This allows for distributed computation of expectation values across multiple devices, as well as the ability to perform checkpoint and resume simulation. Code for saving and (lazily) loading operators so is shown in Sec. 4.

2.5 Noisy simulation

In addition to truncation, another cornerstone of Heisenberg propagation techniques is noisy simulation. The reason for this is that noise can make simulation easier [6]. In particular, depolarizing noise has the effect of damping coefficients, as reviewed in Sec. 3, by causing the coefficients of some terms to go to zero (or below the truncation threshold), reducing the total number of terms in the backpropagated observable and thus reducing the complexity of the simulation.

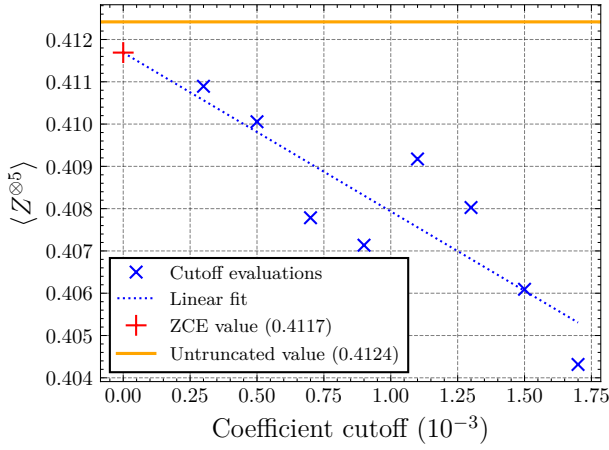


Figure 5: Zero-coefficient-cutoff extrapolation for a five-qubit periodic TFIM circuit evolved for 4 Trotter steps. A linear extrapolation is applied to estimate the untruncated expectation value of $Z^{\otimes 5}$. As can be seen, the extrapolated value is closer to the exact result than the truncated values.

Propaq contains native support for noisy simulation with implementations of depolarizing noise.

Listing 2.5. Native noise models in Propaq.

```
1 import propaq
2 from propaq import UniformNoiseModel
3
4 noise_model = UniformNoiseModel(damping=0.001)
5
6 # Pass into a Pauli/Majorana propagator
7 prop = propaq.PauliPropagator(noise=noise_model)
```

The `UniformNoiseModel` applies single-qubit depolarizing noise to all qubits after every layer of operations. Propaq also defines `GateNoiseModel` which allows the user to specify the noise channel to apply. Beyond this, custom noise channels are supported and can be used with all Propaq propagation features. An example of defining a custom channel for use in noisy simulation is Sec. 4.

2.6 Extrapolation Methods

Both truncation and noisy simulation allow for the implementation of extrapolation methods. In particular, Propaq supports zero-cutoff extrapolation (ZCE) in which the simulation is performed at increasing truncation levels, then a model is fit to the data to predict the zero-cutoff limit behavior of simulation. In Fig. 5, we show that a truncated expectation value can be extrapolated to the zero-cutoff limit for a five-qubit TFIM circuit, improving the simulation accuracy. The

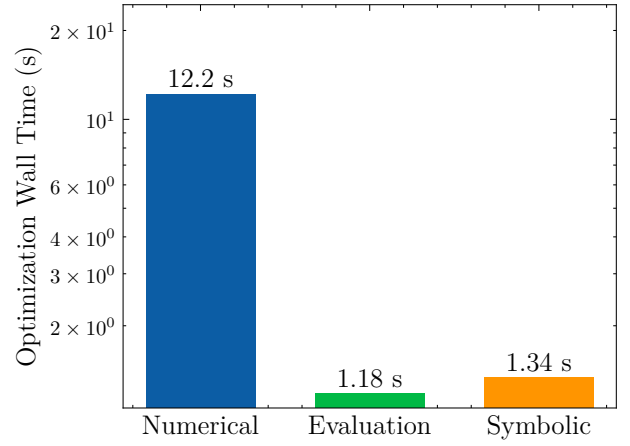


Figure 6: Optimization wall time for depth-two QAOA max-cut ansatz on a 12-vertex, 3-regular graph. 1000 iterations of direct numerical propagation is compared to surrogate model build time (orange) and evaluation time (green). While symbolically simulating the circuit takes 1.34 seconds, after this the total evaluation time for all optimization iterations is 1.18 seconds, providing a significant speedup over optimizing with direct numerical evaluation (12.2 seconds).

API methods for implementing this extrapolation technique are described in Sec. 4.

Since noisy simulation generally makes Heisenberg propagation easier, as mentioned above, it is also natural to consider zero-noise extrapolation [21, 22] in this context. Here, the simulation is performed at increasing noise levels (with decreasing simulation cost), then a model is fit to the data and extrapolated to the zero-noise limit. The API methods for implementing this extrapolation technique are also described in Sec. 4.

2.7 Optimizations

Propaq contains multiple optimizations for performance, both at the software and algorithmic/theory level. While the low-level backend design is described in Sec. 5, we highlight two algorithmic optimizations here. First, tuning (variationally optimizing) gate parameters is a common algorithmic primitive in the variational quantum eigensolver [23], quantum approximate optimization algorithm [24], and other variational quantum algorithms [25–28]. In this setting, circuits need to be simulated many times as a subroutine to optimize the objective function. To mitigate this problem, Propaq supports symbolic/surrogate propagation. In this method, the objective function is computed once with sym-

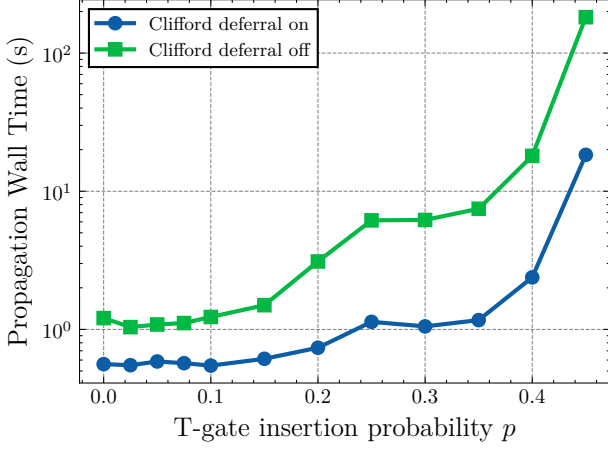


Figure 7: Runtime comparison of $\langle Z \rangle$ calculation with and without Clifford deferral on a 64-qubit, 80 layer Clifford+ T circuit. Each layer of the circuit consists of a random single-qubit Clifford gate on each qubit, followed by a perfect-matching layer of CNOT gates. A T gate is applied to a random qubit with probability p . As shown, Propaq’s Clifford deferral optimization (Sec. 2.7) can lead to a large decrease in runtime, up to around 10x, especially for circuits with high T gate density.

bolic angles, then can be efficiently evaluated for other parameters. While the overhead of the first simulation can be large depending on the problem, this technique can present significant utility when the number of optimization steps is large and renders serial simulation at separate parameters intractable. An example of this is shown in Fig. 6. Here, we fix 1000 iterations for a depth-two QAOA max-cut ansatz on twelve qubits. The total time for optimization numerically (non-symbolic simulation) is 12.2 seconds. While symbolically simulating the circuit the first time takes 1.34 seconds, after this the total evaluation time for all optimization iterations is 1.18 seconds, providing a significant speedup over direct numerical evaluation.

Additionally, Propaq implements another algorithmic optimization for simulating circuits that we call Clifford deferral. In general, for each gate application in the circuit we will have to evaluate two terms, $\cos(\theta)$ and $\sin(\theta)$, where the angle θ is determined by the term and gate (see (9)). This potentially requires allocating and re-merging a large number of terms. For Clifford gates, we can store the adjoint action of the gate in a *stabilizer tableau* [29], and defer the action of a sequence of Clifford gates. Once a non-Clifford gate is encountered, it is pushed through the tableau and applied to all terms in parallel, providing

efficiency gains. To illustrate this, we perform a timing study, comparing the performance of Propaq with and without Clifford deferral on random Clifford+ T circuits. The results, shown in Fig. 7, demonstrate that deferring Clifford gates can confer significant performance gains for circuits with high densities of Clifford gates.

3 Background

3.1 Heisenberg Propagation

We briefly review the theory of Heisenberg propagation [2, 11, 12, 30]. Consider a quantum circuit

$$\mathcal{C} := U_L \dots U_2 U_1 \quad (4)$$

consisting of L gates, an observable $\mathcal{O}$, and a quantum state $|\Psi_0\rangle$. The expectation value of the observable with respect to the state after the circuit is applied is given by

$$\mathbb{E}[\mathcal{O}] := \langle \Psi_0 | \mathcal{C}^\dagger \mathcal{O} \mathcal{C} | \Psi_0 \rangle = \text{Tr} \left[\mathcal{O} \left(\mathcal{C} \rho \mathcal{C}^\dagger \right) \right] \quad (5)$$

with

$$\rho := |\Psi_0\rangle\langle\Psi_0|. \quad (6)$$

This is the Schrödinger picture of quantum mechanics in which the state evolves in time and the observable is fixed. Through the cyclicity of the trace, we can equivalently write this as

$$\mathbb{E}[\mathcal{O}] = \text{Tr} \left[\left(\mathcal{C}^\dagger \mathcal{O} \mathcal{C} \right) \rho \right] \quad (7)$$

to recover the Heisenberg picture, where the observable is *back-propagated* through the circuit and the state remains fixed. By representing the observable in a basis of operators $\{\sigma_i\}$, we can write

$$\mathcal{O} = \sum_i c_i \sigma_i$$

where c_i are the coefficients of the observable in the operator basis. The action of the circuit is precisely the adjoint action of each layer on the basis elements, which can be computed via the commutation relations of the operator basis. After parameterizing each gate as

$$U_i = \exp\{-i\theta\sigma_i/2\}, \quad (8)$$

the action of the gate on a basis element σ_j is given by

$$\sigma_j \mapsto \begin{cases} \sigma_j & [\sigma_i, \sigma_j] = 0 \\ \cos(\theta)\sigma_j + i\sin(\theta)\sigma'_i & [\sigma_i, \sigma_j] \neq 0 \end{cases} \quad (9)$$

where σ'_i is a new basis element generated by the commutator $[\sigma_i, \sigma_j]$. Because of this, the number of terms in the back-propagated observable will generally grow exponentially with the number of gates in the circuit, and it is often necessary to truncate the back-propagated observable to a manageable number of terms. This is generally done by discarding terms with small coefficients or high weight.

3.2 Noisy Heisenberg Propagation

Heisenberg propagation with noise can lessen simulation complexity [6] and thus presents a common application. Here, we are interested in the computation of (7) with noise applied to the state. That is, our goal is to estimate

$$\mathbb{E}[\mathcal{O}] := \text{Tr}[\mathcal{E}(U\rho U^\dagger)\mathcal{O}] \quad (10)$$

where $\mathcal{E}$ is a quantum channel (completely positive, trace preserving map). Naturally, there are many such maps $\mathcal{E}$ defined by their action and when/where they act during the quantum computation. Most generally, a quantum channel is defined by its Kraus operators $\{K_i\}_{i=1}^m$ via

$$\mathcal{E}(\rho) = \sum_i K_i \rho K_i^\dagger, \quad (11)$$

with the only restriction on K_i being the completeness relation

$$\sum_i K_i^\dagger K_i = I. \quad (12)$$

Two typical models that have been studied are single-qubit depolarizing noise

$$\mathcal{E}_p(\rho) := (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z) \quad (13)$$

where $0 \leq p \leq 1$ and X, Y , and Z are the Pauli matrices applied (1) after every parallel layer of gates in the circuit, or (2) after every gate in the circuit. These have been studied in [6] because they model quantum hardware relatively well but are still tractable to work with analytically. We remark that Propaq implements (1) as `UniformNoiseModel`, as shown in Sec. 4. Consider a single-qubit state ρ acted on by a depolarizing channel (11). Our goal is to evaluate

$$\text{Tr}[\mathcal{E}_p(\rho)\mathcal{O}]. \quad (14)$$

Using the Kraus operator representation (11) and simple properties of the trace, we may write

$$\text{Tr}[\mathcal{E}(\rho)\mathcal{O}] = \text{Tr}\left[\left(\sum_i K_i \rho K_i^\dagger\right)\mathcal{O}\right] \quad (15)$$

$$= \sum_i \text{Tr}\left[K_i \rho K_i^\dagger \mathcal{O}\right] \quad (16)$$

$$= \sum_i \text{Tr}\left[\rho K_i^\dagger \mathcal{O} K_i\right] \quad (17)$$

$$= \text{Tr}\left[\rho \left(\sum_i K_i^\dagger \mathcal{O} K_i\right)\right] \quad (18)$$

$$=: \text{Tr}[\rho \mathcal{E}^\dagger(\mathcal{O})]. \quad (19)$$

In the last step, we defined the adjoint (dual) channel $\mathcal{E}^\dagger$ with Kraus operators $\{K_i^\dagger\}$. Certain channels, notably the single-qubit depolarizing channel (13), are self-adjoint, meaning that $K_i^\dagger = K_i$ for all i and so $\mathcal{E}^\dagger = \mathcal{E}$.

In Pauli propagation,

$$\mathcal{O} = \sum_i c_i P_i \quad (20)$$

where P_i are n -qubit Pauli operators of the form

$$P = s X_1^{x_1} X_2^{x_2} \cdots X_n^{x_n} Z_1^{z_1} Z_2^{z_2} \cdots Z_n^{z_n}. \quad (21)$$

Here, $s \in \mathbb{C}$ is a phase, X and Z are the single-qubit Pauli operators defined by

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (22)$$

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad (23)$$

and $x_i, z_i \in \{0, 1\}$. Note that $Y := iXZ$ can be generated up to phase from the product of X and Z , and that $X^2 = Y^2 = Z^2 = I$. Since $\mathcal{P} := \{I, X, Y, Z\}$ is a basis for any single-qubit operator, $\mathcal{P}_n := \mathcal{P}^{\otimes n}$ is a basis for any n -qubit operator.

Consider a single-qubit Pauli P under a single-qubit depolarizing channel $\mathcal{E}_p$. By definition, we have

$$\mathcal{E}_p(P) = (1-p)P + \frac{p}{3}(XPX + YPY + ZPZ). \quad (24)$$

For any non-identity Pauli $P \in \{X, Y, Z\}$, it is easy to verify that

$$XPX + YPY + ZPZ = -P. \quad (25)$$

Thus, the action of the depolarizing channel on a non-identity Pauli can be written

$$\mathcal{E}_p(P) = (1-p)P + \frac{p}{3}(-P) = \left(1 - \frac{4}{3}p\right)P. \quad (26)$$

For $P = I$, we have that

$$\mathcal{E}_p(I) = I. \quad (27)$$

Therefore, when propagated through a channel, non-identity Paulis pick up a *damping factor* of

$$e^{-\gamma} := (1 - 4p/3), \quad (28)$$

while the identity is unchanged.

For a tensor product channel acting on an n -qubit Pauli string

$$P \equiv P_1 \otimes \cdots \otimes P_n \mapsto \mathcal{E}_p(P_1) \otimes \cdots \otimes \mathcal{E}_p(P_n), \quad (29)$$

we see that

$$P \equiv P_1 \otimes \cdots \otimes P_n \mapsto e^{-\gamma w[P]} P \quad (30)$$

where $w[P]$ is the weight (number of non-identity Paulis) of Pauli P .

In Majorana propagation,

$$\mathcal{O} = \sum_i c_i M_i \quad (31)$$

where M_i are n -qubit Majorana operators (often called Majorana monomials in this context) of the form

$$M = s \gamma_1^{b_1} \gamma_2^{b_2} \cdots \gamma_{2n-1}^{b_{2n-1}} \gamma_{2n}^{b_{2n}}. \quad (32)$$

Here, $s \in \mathbb{C}$ is a phase, the Majorana operators are defined by

$$\gamma_{2j-1} := \left(\prod_{k=1}^{j-1} Z_k \right) X_j \quad (33)$$

$$\gamma_{2j} := \left(\prod_{k=1}^{j-1} Z_k \right) Y_j, \quad (34)$$

and $b_i \in \{0, 1\}$.

Under a tensor product of single-qubit channels

$$\mathcal{E}(M) := (\mathcal{E}_1 \otimes \cdots \otimes \mathcal{E}_n)(M), \quad (35)$$

we can express the Majorana M in the Pauli basis $\mathcal{P}(M)$ via (33) - (34) to obtain a similar mapping to (30), namely

$$\mathcal{E}_p(\mathcal{P}(M)_1) \otimes \cdots \otimes \mathcal{E}_p(\mathcal{P}(M)_n) = e^{-\gamma w[\mathcal{P}(M)]} M. \quad (36)$$

The only difference relative to (36) is that in (30) we must use the *Pauli weight* $w[\mathcal{P}(M)]$ instead of the Majorana weight $w[M]$. Since Majoranas are nonlocal, a single Majorana operator can have high Pauli weight, e.g. $w[\mathcal{P}(\gamma_{2n})] = n$, leading to different damping factors in noisy simulation.

3.3 Surrogate/Symbolic Propagation

Consider a parameterized quantum circuit $\mathcal{C}(\Theta)$, an observable $\mathcal{O}$, and an initial state $|\Psi_0\rangle$. It is often of interest to compute the expectation value of the observable with respect to the evolved state as a function of the parameters Θ :

$$\mathbb{E}[\mathcal{O}](\Theta) = \langle \Psi_0 | \mathcal{C}(\Theta)^\dagger \mathcal{O} \mathcal{C}(\Theta) | \Psi_0 \rangle \quad (37)$$

This is particularly relevant for variational quantum algorithms and quantum machine learning applications. Although one can compute the expectation value for each parameter setting Θ using numerical propagation, this can become computationally expensive for large circuits and many parameter settings. The parameters do not affect the circuit structure, rather they only affect the coefficients of the back-propagated operator. Therefore, rather than numerically propagating the operator for each Θ , one can *symbolically* propagate the operator once to obtain

$$f : \Theta \mapsto \mathbb{E}[\mathcal{O}](\Theta) := \sum_i c_i(\Theta) \text{Tr}(P_i \rho_0) \quad (38)$$

i.e. a function that maps the parameters to the expectation value of the observable. Terms that are structurally negligible can be pruned from the back-propagated operator, resulting in a *surrogate model* that can be evaluated for any parameter setting Θ . Propaq implements surrogate propagation and provides an API for saving, loading, and evaluating surrogate models. The details of implementation and usage are described in Sec. 4.8. It is worth noting that surrogate propagation is significantly more expensive than numerical propagation, as a result of having to track the symbolic coefficient tree and perform symbolic algebra, and the method should be used when many energy evaluations are required for parameter tuning such that fast subsequent energy evaluation outweighs the initial cost, as shown in the example of Fig. 6.

4 API and Usage

In this section, we provide a “getting started” guide with Propaq, starting with installation and proceeding through most common uses. This section is designed to be self-contained and provide more details on the features highlighted in Sec. 2, as well as show additional features.

4.1 Installation and Dependencies

Propaq can be installed on Linux, Windows and Mac via PyPI using the following command:

Listing 4.1. Installation of Propaq via PyPI.

```
pip install propaq
```

Propaq is natively compatible with Qiskit circuits, and this is a dependency of the library. Compatibility with Cirq [31], FFSIM [32], OpenFermion [33], and Quimb [34] (for hybrid Schrödinger-Heisenberg simulation with matrix product states) are optional extras that can be installed via the following command:

Listing 4.2. Installation of Propaq with optional extras.

```
pip install propaq[cirq,ffsim,openfermion,hybrid]
```

The source code for Propaq is available on GitHub [35].

4.1.1 Building for a specific CPU

The wheels distributed on PyPI are built to be generic and compatible with a wide range of CPUs across different architectures. Several hot paths in the propagation engine can be autovectorized by the compiler, and therefore benefit from CPU-specific optimizations. Therefore, for users who want to maximize performance on known CPU architecture, Propaq can be built from source using a Rust toolchain:

Listing 4.3. Building Propaq for the host CPU.

```
git clone https://github.com/hkbelagali/propaq
cd propaq
maturin build --release
pip install target/wheels/propaq-*.whl
```

This build will generally only work on the exact CPU architecture it was built on, therefore building in cluster environments must be done on the same target node that will be used for running the simulations.

4.2 Basic Usage

The following code block shows the basic usage of Propaq to simulate a circuit.

Listing 4.4. Basic usage of Propaq.

```
1 import qiskit
2
3 from propaq import PauliCircuit, PauliTermSum,
   ↪ PauliPropagator, UniformNoiseModel,
```

Listing 4.4. (continued)

```
   ↪ CoefficientTruncator
4
5 qc: qiskit.QuantumCircuit = ...
6 obs: qiskit.SparsePauliOp = ...
7 initial_state: int = ...
8
9 # Convert to Propaq circuit and observable.
10 pc = PauliCircuit.from_qiskit(qc)
11 pts = PauliTermSum.from_sparse_pauli_op(obs)
12
13 # Construct a propagator.
14 prop = PauliPropagator(
15     noise=UniformNoiseModel(damping=0.001),
16     truncation=[CoefficientTruncator(1e-6)],
17     n_threads=16,
18     progress_bar=True
19 )
20
21 # Compute the expectation value.
22 result = prop.expectation_value(
23     observable=pts,
24     circuit=pc,
25     initial_state=initial_state
26 )
27 result.expectation_value # Type: float
```

Here, we assume that the user has a quantum circuit defined as a `qiskit.QuantumCircuit`, an observable expressed as a `qiskit.SparsePauliOp`, and an initial bitstring represented as an integer (e.g. 0 for the all $|0\rangle^{\otimes n}$ state in the computational basis). After, a `PauliPropagator` is constructed with various options for noise, truncation, number of threads to use, and whether or not to show simulation progress. These options are used when the `expectation_value` method is called to compute the expectation value of the observable in the circuit and initial state.

One can use the analogous method `from_cirq` to parse a Cirq circuit. Additionally, the `MajoranaTermSum`, `MajoranaCircuit`, and `MajoranaPropagator`, along with their conversion methods from Qiskit, Cirq, OpenFermion, and ffsim can be used to perform Majorana propagation. Note that these require optional dependencies as described in Sec. 4.1.

4.3 Truncation and Noise Policies

4.3.1 Built-in Truncation Policies

Propaq implements truncation policies which can be used to control the growth of the operator during propagation. The built-in truncation policies include `CoefficientTruncator`, `WeightTruncator` (numeric), and `FrequencyTruncator` (symbolic). The propagator objects accept a list of truncation policies, which are applied in the order pro-

vided. Users can also specify a `TermBudget`, which suppresses all other lossy truncation policies until the operator has at least the given number of live terms, keeping propagation exact until it has had room to grow. For example, the following list of truncation policies says to truncate terms with coefficients below 10^{-6} and weight above 5, but keep propagation exact below 100 terms.

Listing 4.5. Truncation policies in Propaq.

```
1 truncation = [
2     CoefficientTruncator(1e-6),
3     WeightTruncator(5),
4     TermBudget(min_terms=100),
5 ]
```

This list of truncation policies can then be provided to a propagator, e.g. the `PauliPropagator` from Listing 4.4, and then used in simulation.

4.3.2 Built-in Noise Models

Propaq provides uniform depolarizing noise for both Pauli and Majorana propagation through the `UniformNoiseModel` class. This applies a uniform depolarizing channel to each qubit after every layer in the circuit.

Listing 4.6. Uniform depolarizing noise.

```
1 from propaq import UniformNoiseModel
2
3 # Generate a model scaling each term by
4   ↪ exp(-0.001 * weight[P])
5 noise_model = UniformNoiseModel(damping=0.001)
6
7 # Pass into a Pauli/Majorana propagator
8 prop = PauliPropagator(noise=noise_model)
```

Since the propagation engine is generic, the `UniformNoiseModel` can be used with any propagator. For example, it can be used directly with Weyl-Heisenberg propagation (see Appendix A) and naturally generalizes to a qudit depolarizing channel.

4.3.3 Custom Noise and Truncation Policies

While Propaq contains native support for depolarizing noise, custom noise models are supported. This can be done by inheriting from `GateNoiseModel`, as shown below. In this example, we implement a channel for depolarizing noise

$$\mathcal{E}(\rho) = (1 - p)\rho + pZ\rho Z \quad (39)$$

with $p = (1 - e^{-\gamma})/2$.

Listing 4.7. Custom noise in Propaq.

```
1 import math
2 from propaq.noise import GateNoiseModel
3
4 class DephasingGateNoise(GateNoiseModel):
5     X_MASK = 0x5555555555555555
6
7     def __init__(self, gamma: float) -> None:
8         self.gamma = gamma
9
10    def damping_factor_term(self,
11        basis_kind: int,
12        words: list[int],
13        n_units: int,
14        weight: int
15    ) -> float:
16        x_count = sum(
17            bin(w & self.X_MASK).count("1")
18            for w in words
19        )
20        return math.exp(-self.gamma * x_count)
```

As shown, the necessary method to implement is `damping_factor_term` which returns the numerical value to damp coefficients of backpropagated observables by, depending on the basis and observable. Other custom channels can be defined in this manner and subsequently used with propagators in the normal way.

Since the backend of Propaq is in Rust, custom noise in Python can be sped up by defining them directly in Rust. For this reason, Propaq features a plugin application binary interface (ABI), through which users can implement their own noise and truncation policies in Rust, C, or ahead-of-time (AOT) compiled Julia. This method works by accepting a shared library (`.so` on Linux, `.dylib` on Mac, and `.dll` on Windows) that specifies its permissions and implements the noise or truncation policy. As an example, we demonstrate how to implement a noise plugin in C in Appendix B, which defines `uniform_noise.c`. After this, the shared library is built so that it is available as `libuniform_noise.so`. Then, it can be loaded and used in Propaq as follows:

Listing 4.8. Using a custom noise plugin in Propaq.

```
1 from propaq.noise import NativeNoiseModel
2 from propaq.truncation import NativeTruncator
3
4 damping = 0.001
5
6 # Load the shared library, and pass in damping
7   ↪ as a JSON string
8 noise_model = NativeNoiseModel(
9     'libuniform_noise.so',
10    config=f'{{"damping": {damping}}}'
11 )
12
13 # Use like any other noise model
14 prop = PauliPropagator(noise=noise_model)
```

Listing 4.8. (continued)

```

14
15 # Custom truncation policy
16 truncator = NativeTruncator(
17     'libtruncator.so',
18     config=f'{"...": ...}'
19 )
20
21 prop = MajoranaPropagator(truncation=truncator)

```

We remark that Propaq employs a permission system for its noise and truncation plugins. This ensures that the plugin has access to the necessary data and resources, while allowing the engine to perform optimizations and parallelization depending on the access level. For example, the action of a noise plugin that only requires the weight (such as uniform depolarizing noise) can be pre-computed once at the beginning of the propagation. This virtually eliminates all overhead of the plugin, and allows for efficient parallel execution. In contrast, a plugin that requires access to full term's structure generally cannot be pre-computed, and requires careful design to prevent race conditions and ensure reproducibility. The permissions and details for implementing plugins are described in the Propaq documentation [35].

4.4 Arbitrary basis propagation

Propaq supports propagation in arbitrary bases through its Python API. The user can define a custom basis representation and algebraic relations by subclassing the `AbstractTerm`, `DictTermSum`, and `AbstractPropagator` classes. This allows the user to utilize extrapolation methods and term I/O for eligible custom bases. As an example, we study a single spin- S system, which lives in a Hilbert space of dimension $d = 2S + 1$ with commutation relation

$$[S_a, S_b] = i\epsilon_{abc}S_c. \quad (40)$$

For $S = 1/2$, this reduces to the Pauli algebra. Note that this is not a complete operator basis for $S > 1/2$. We can define a single parameterized gate $U_{\mathbf{n}}(\theta)$ for a rotation about an arbitrary axis $\mathbf{n}$, given by

$$U_{\mathbf{n}}(\theta) = \exp\{-i\theta(\mathbf{n} \cdot \mathbf{S})\} \quad (41)$$

Then, via Wigner's theorem, we get the closed-form for every rotation S ,

$$U_{\mathbf{n}}^\dagger(\theta)S_aU_{\mathbf{n}}(\theta) = \sum_b R_{ab}(\mathbf{n}, \theta)S_b \quad (42)$$

where $R_{ab}(\mathbf{n}, \theta) \in \text{SO}(3)$ is a rotation matrix given by

$$R(\mathbf{n}, \theta) = \cos(\theta)I + \sin(\theta)[\mathbf{n}]_{\times} + (1 - \cos(\theta))\mathbf{nn}^\top \quad (43)$$

where $[\mathbf{n}]_{\times}$ is the cross-product matrix of $\mathbf{n}$. The overlap between a spin coherent state $|\mathbf{n}\rangle = U_{\mathbf{n}}(\theta)|S, S\rangle$ and a spin operator S_a can be computed as

$$\langle \mathbf{n} | S_a | \mathbf{n} \rangle = S \cdot n_a \quad (44)$$

In the following code snippet, we demonstrate how to implement these spin operators in Propaq by subclassing the appropriate methods.

Listing 4.9. Subclassing Propaq for a single spin- S particle.

```

1 from dataclasses import dataclass
2 import numpy as np
3 from propaq.datatypes import AbstractTerm,
4     DictTermSum
5 from propaq.propagators import
6     AbstractPropagator
7
8 @dataclass(frozen=True, slots=True)
9 class SpinTerm(AbstractTerm):
10     axis: int # 0, 1, 2 for Sx, Sy, Sz
11     spin: float # S
12
13     @property
14     def weight(self) -> int: # 3. from 2.2
15         return 1
16
17     @property
18     def n_units(self) -> int:
19         return 1
20
21     def commutes_with(self, other: "SpinTerm")
22     -> bool: # 2. from 2.2
23         return self.axis == other.axis
24
25     def __matmul__(self, other: "SpinTerm") ->
26     tuple[complex, "SpinTerm"]: # 1. from 2.2
27         raise NotImplementedError
28
29     def trace_with_diag_state(self, diag_state)
30     -> complex: # 4. from 2.2
31         return self.spin * diag_state[self.axis]
32
33     def to_bytes(self) -> bytes: # 6. from 2.2
34         return bytes([self.axis, round(2 *
35             self.spin)])
36
37     @classmethod
38     def from_bytes(cls, data: bytes, n_units:
39     int) -> "SpinTerm": # 6. from 2.2
40         axis, doubled_spin = data[0], data[1]
41         return cls(axis, doubled_spin / 2)
42
43     def __hash__(self) -> int: # 6. from 2.2
44         return hash((self.axis, self.spin))
45
46     def __eq__(self, other: object) -> bool: #
47         from 2.2
48         return isinstance(other, SpinTerm) and
49             self.axis == other.axis and self.spin ==
50             other.spin

```

Listing 4.9. (continued)

```

42 class SpinTermSum(DictTermSum[SpinTerm]):
43     term_type = SpinTerm # enables file I/O
44
45 class SpinRotation:
46     def __init__(self, axis: tuple[float,
47     ↪ float, float], angle: float):
48         n = np.asarray(axis) /
49     ↪ np.linalg.norm(axis)
50         cross = np.array([[0, -n[2], n[1]],
51     ↪ [n[2], 0, -n[0]], [-n[1], n[0], 0]])
52         self.R = np.cos(angle) * np.eye(3) +
53     ↪ np.sin(angle) * cross + (1 -
54     ↪ np.cos(angle)) * np.outer(n, n)
55
56 class SpinPropagator(AbstractPropagator):
57     term_sum_type = SpinTermSum
58
59     def apply_gate(self, term: SpinTerm, coeff:
60     ↪ complex, rotation: "SpinRotation"): # 5.
61     ↪ from 2.2
62         for b in range(3):
63             weight = rotation.R[term.axis, b]
64             if weight:
65                 yield SpinTerm(b, term.spin),
66     ↪ coeff * weight

```

Although the matrix multiplication method is not required for the spin basis, it is necessary to implement to satisfy the `AbstractTerm` interface. We also note that custom bases are not currently routed through the Rust backend, and are therefore slower than the built-in Pauli and Majorana bases by a non-trivial constant factor. We are actively working on extending support for custom bases into the Rust backend.

4.5 Custom Gate Registration

Heisenberg propagation is a general procedure that can be applied to any gate, provided that it can be parameterized in terms of basis generators and angles. It is impractical to implement every possible gate in the core engine, therefore Propaq decomposes gates into its native basis using quantum compilation techniques. Although this ensures that any gate can be accurately represented, the decompositions are not necessarily optimal, and can lead to larger-than-necessary operator growth. For cases where the user has a custom gate that is not in the native basis with a known, optimal decomposition, Propaq allows the user to register the gate in terms of its generator representation. Propaq will validate the user-supplied generator representation by comparing it to its native decomposition. If the two representations are equivalent, the gate can be used in the propagation engine in its optimized form. As an example, we demonstrate how to register a CX

gate in Propaq using its generator representation:

Listing 4.10. Registering a CX gate in Propaq.

```

1 from propaq.circuits import PauliCircuit,
  ↪ pauli_rotation_generator,
  ↪ register_qiskit_gate
2
3 def cnot_terms(
4     instr: qiskit.circuit.Instruction,
5     q_indices: tuple[int, int],
6     width: int,
7     rep: GateRep
8 ):
9     i, j = q_indices # control, target
10    n_qubits = rep.qubits_in_width(width)
11
12    # Get the string repr of a Pauli
13    def label(axis_i, axis_j):
14        chars = ["I"] * n_qubits
15        if axis_i:
16            chars[n_qubits - 1 - i] = axis_i
17        if axis_j:
18            chars[n_qubits - 1 - j] = axis_j
19        return "".join(chars)
20
21    # Build the generator terms
22    terms = []
23    for axis_i, axis_j, coeff in (
24        ("Z", None, math.pi / 2),
25        (None, "X", math.pi / 2),
26        ("Z", "X", -math.pi / 2),
27    ):
28        gen, unit = pauli_rotation_generator(
29            rep,
30            label(axis_i, axis_j)
31        )
32        terms.append((gen, coeff * unit))
33    return [terms]
34
35 # Register the qiskit gate
36 register_qiskit_gate("cx", cnot_terms)

```

For certain gates, gate registry can yield significant performance improvements. An example is shown in Fig. 8 where custom decomposition registered with Propaq achieves a 2-5x speedup when benchmarking the wall clock time of propagation for a ten-qubit non-Clifford brickwork exchange ansatz.

4.6 Logging, Storage, and I/O

4.6.1 Logging

Propaq supports logging of propagation statistics and runtime diagnostics. These features are provided by `Logger` objects.

Listing 4.11. Logging in Propaq.

```

1 from propaq import Logger
2
3 # Save to propagation.log, and log every 5 gates.
4 logger = Logger(
5     filename='propagation.log',
6     log_every=5,

```

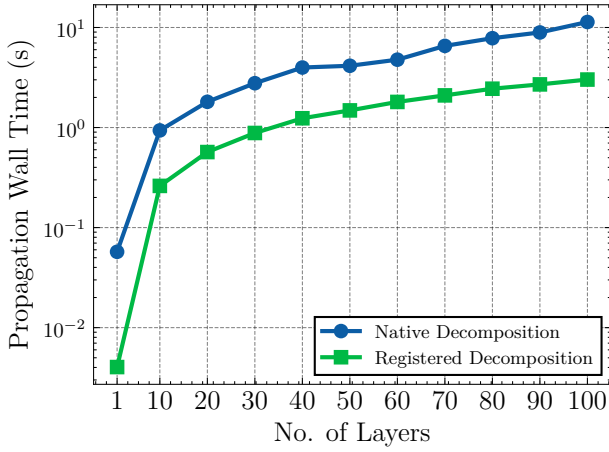


Figure 8: Propagation runtime for a 10-qubit non-Clifford brickwork exchange ansatz for a Propaq’s native decomposition, and a custom decomposition registered with Propaq.

Listing 4.11. (continued)

```
7 )
8 prop = PauliPropagator(logger=logger)
```

This writes information to the log file at regular intervals defined by `log_every` in a JSONL format. The data can be parsed and analyzed using the `LogParser` class, which returns trajectories of each logged quantity.

Listing 4.12. Parsing logs in Propaq.

```
1 from propaq import LogParser
2
3 log = LogParser('propagation.log')
4 log.terms_discarded # list[int]
```

Additional logged quantities include `discarded_coeff_l1`, `discarded_coeff_max`, `ms_per_gate`, `elapsed_ms`, `gate_events`, `terms_before`, `terms_after`, and `truncation_events`, among others defined in Propaq’s documentation [35].

4.6.2 Storage and I/O

Storing propagated operators is a common requirement for many applications and downstream analysis. Propaq provides a simple interface for storing and loading propagated operators in a binary format.

Listing 4.13. Storing and loading propagated operators in Propaq.

```
1 from propaq import PauliTermSum
2
3 # Propagate and save a term sum to disk.
```

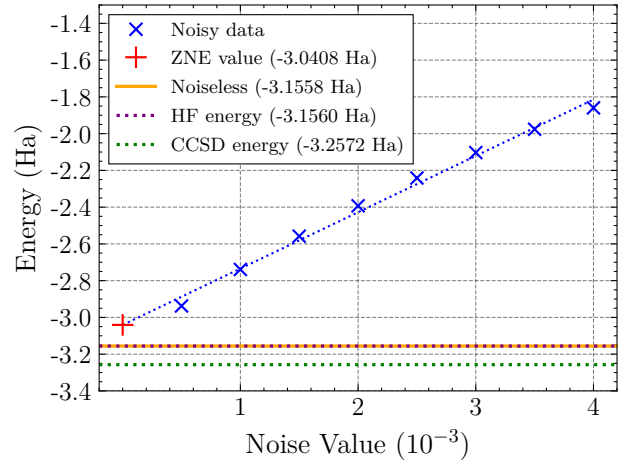


Figure 9: Zero-noise extrapolation of a 2-layer unitary cluster Jastrow (UCJ) ansatz for a $(\text{H}_2)_3$ molecule. A uniform depolarizing noise channel is applied after each layer of the ansatz, and a linear extrapolation is performed to estimate the expectation value.

Listing 4.13. (continued)

```
4 prop = PauliPropagator(...)
5 prop.propagate(..., filename='obs.gz')
6
7 # Load a term sum from disk.
8 termsum = PauliTermSum.from_file('obs.gz')
```

Term sums can also be saved to disk through via `termsum.save('obs.gz')`. These operations are *eager* in memory, and therefore impractical for very large operators. Therefore, we also provide a lazy interface for loading and merging operators. This enables distributed workflows for propagation. Users can partition an observable and run on different devices, store the results in disk, and then merge the results into a single operator later for analysis.

Listing 4.14. Lazy loading and merging of propagated operators in Propaq.

```
1 from propaq import PauliTermStreamer
2
3 streamer = PauliTermStreamer.from_file('obs.gz')
4
5 # Or: merge multiple operators into one
6 files = ['obs1.gz', 'obs2.gz', 'obs3.gz']
7 obs = PauliTermSum()
8 for f in files:
9     obs.merge_from_file(
10         PauliTermStreamer.from_file(f)
11     )
```

Merging using a streamer ensures that the operator is not first materialized and then merged, which redundantly increases memory usage.

4.7 Extrapolation Methods

Propaq provides classes and methods to implement extrapolation methods for zero-cutoff and zero-noise limit behavior of quantum circuits, which are detailed in the following section. Propaq also allows users to implement extrapolators for custom noise and truncation policies by subclassing `ZeroNoiseExtrapolator` and `ZeroCutoffExtrapolator`, the details of which are mentioned in Propaq’s documentation [35].

4.7.1 Zero-Cutoff Extrapolation

An example result of zero-cutoff extrapolation improving the accuracy of expectation values was shown in Fig. 5. The following code block shows the general structure for using zero-cutoff extrapolation in Propaq.

Listing 4.15. Zero-cutoff extrapolation in Propaq.

```
1 from propaq import CoefficientCutoffExtrapolator
2 from propaq import CoefficientTruncator
3
4 circuit: PauliCircuit = ...
5 observable: PauliTermSum = ...
6 prop: PauliPropagator = PauliPropagator(
7     truncation=[CoefficientTruncator(1e-6)]
8 )
9
10 cce = CoefficientCutoffExtrapolator(
11     fitting_fn = lambda x, a, b: a + b * x,
12     cutoff_values=[1e-6, 1e-5, 1e-4, 1e-3]
13 )
14 result_cce = cce.run(
15     propagator=prop,
16     circuit=circuit,
17     observable=observable,
18     initial_state=0,
19 )
```

As shown, zero-cutoff extrapolation can be easily implemented by simply defining the fit function `fitting_fn` and set of cutoff values to run. We remark that Propaq also defines a `WeightCutoffExtrapolator` which performs truncation based on observable weight. The usage for this extrapolator is identical to the above.

4.7.2 Zero-Noise Extrapolation

In addition to zero-cutoff extrapolation, Propaq also implements zero-noise extrapolation (ZNE). While ZNE has been implemented in several quantum error mitigation packages [36, 37], it is natural to consider ZNE for noisy Heisenberg propagation because, as discussed in Sec. 3, it reduces simulation complexity at larger noise levels. For this reason Propaq natively implements ZNE

through the `ZeroNoiseExtrapolator` class. An example of applying ZNE with Propaq is shown below.

Listing 4.16. Zero-noise extrapolation in Propaq.

```
1 from propaq import ZeroNoiseExtrapolator
2
3 circuit: PauliCircuit = ...
4 observable: PauliTermSum = ...
5 noise = UniformNoiseModel(damping=0.00)
6 prop = PauliPropagator(noise=noise)
7
8 zne = ZeroNoiseExtrapolator(
9     fitting_fn = lambda x, a, b: a + b * x,
10    noise_values=[0.005, 0.010, 0.015, 0.020]
11 )
12 result = zne.run(
13     propagator=prop,
14     circuit=circuit,
15     observable=observable,
16     initial_state=0,
17 )
```

Example results are shown in Fig. 9 for a two-layer unitary cluster Jastrow ansatz for a hydrogen chain. While both the bias and variance of ZNE is strongly noise and problem dependent, and still largely an active area of research, Propaq’s implementation allows researchers to easily use and analyze these techniques in the context of Heisenberg propagation.

4.8 Variational Algorithms

Symbolic propagation allows for the construction of surrogate models for variational algorithms and parameter sweeps. The `SurrogatePropagator` class provides the interface necessary to run these algorithms, porting from Qiskit and Cirq objects.

Listing 4.17. Surrogate propagation in Propaq.

```
1 # Construct a parameterized Qiskit circuit
2 theta = ParameterVector('theta', 6)
3 qc = QuantumCircuit(3)
4
5 qc.x(0)
6 qc.x(2)
7
8 it = 0
9 for layer in range(2):
10     for q in range(3):
11         qc.rz(theta[it], q)
12         it += 1
13
14 # Build the surrogate model
15 obs : PauliTermSum = ...
16 circuit = SurrogatePauliCircuit.from_qiskit(qc)
17
18 model = PauliSurrogatePropagator().build(obs,
19     ↪ circuit, initial_state=0)
20
21 # Wrap the model for ease of use
22 variational_model = VariationalSurrogateModel(
```

Listing 4.17. (continued)

```

22     model, circuit.parameter_sources,
    ↪ circuit.qiskit_parameters
23 )
24
25 # Optimize using scipy.minimize
26 def cost(x):
27     return variational_model.evaluate(x)
28
29 result = minimize(cost, x0=np.random.rand(6),
    ↪ method='COBYLA', options={'maxiter': 1000})

```

4.9 Hybrid Simulation

The hybrid Schrödinger–Heisenberg simulation is provided through the `hybrid_expectation_value` method. We provide a code snippet demonstrating how to use this method, assuming the user has a quimb `MatrixProductState` object, and a `PauliTermSum` back-propagated observable.

Listing 4.18. Hybrid Schrödinger–Heisenberg simulation in Propaq.

```

1  from quimb.tensor import MatrixProductState
2  from propaq import PauliTermSum,
    ↪ hybrid_expectation_value
3
4  # Forward-evolved state.
5  mps: MatrixProductState = ...
6
7  # Backward-evolved observable.
8  observable: PauliTermSum = ...
9
10 # Compute the expectation value
11 ev = hybrid_expectation_value(observable, mps)

```

Here, the `mps` represents the forward simulation of the initial state to the circuit bipartition, and the `obs` represents the backward simulation of the observable to the bipartition. Of course, back-propagating `obs` to this point can be done with any propagation methods in Propaq previously described.

5 Design and Implementation

Propaq is implemented in Rust, with `rayon` used for multithreading and `pyo3` used to expose the Rust API to Python. This section describes the design and implementation of Propaq, including its data types and parallelization. A general trend in the design of Propaq is to use different data types for the Python frontend and the Rust backend, with fast conversions between the two. This gives Propaq a simple Python interface, and makes it compatible with most major quantum software packages written in Python, in addition to ensuring high performance. Python

and Rust differ in their design patterns, necessitating different data types and interfaces for the two languages, described in this section. We remark that several design decisions were adopted from MONOPROP [16] — notably `BasisString`, `OperatorIndex`, and `InvertedIndex` data types — while other design elements, previously mentioned features, and the Rust backend/Python frontend, were uniquely chosen.

5.1 Term Storage

The nature of Heisenberg propagation demands a data structure that can compactly represent enormous numbers of operators, and allow for efficient computation of their algebraic relations. The symplectic representation of Pauli and Majorana operators is a natural choice, where the operators are represented as bitmasks, and the commutation relations become bitwise operations. Consistent with Rust’s design philosophy, we separate the term storage (`BasisString`) from term behavior (`PauliAlgebra/MajoranaAlgebra`). This allows the engine to be generic over the basis, and promotes the implementation of new bases in the future. Propaq utilizes three main data types for representing operators: `Bitset`, `BasisString`, and `OperatorIndex`. The second two are integrated with the dynamically-sized `Bitset` data type for `pyo3` boundary crossing.

5.1.1 Bitset

We implement a `Bitset` data type that can represent an arbitrary number of bits, and provide efficient implementations of the bitwise operations required for Heisenberg propagation. The `Bitset` data type is a contiguous array of 64-bit unsigned integers, where each bit represents a mode (qubit or Majorana mode). It allocates 4 integers by default, which supports up to 256/128 qubits for Pauli/Majorana operators without heap allocation.

5.1.2 BasisString<W>

The dynamic size of `Bitset` makes it convenient for traveling across the `pyo3` boundary. However, since the width of the bitmask is unknown at compile time, the compiler is limited in the optimizations it can perform on the bitwise operations, which is important for hot loop operation performance. Therefore, we implement

a `BasisString` data type that dispatches fixed-size operator representations for the Rust backend based on the system size. For example, if the system has 14 qubits, the `BasisString<W>` will dispatch with $W = 1$ 64-bit word, the next-largest available width. The qubit thresholds are 32, 64, 128, up to 2048 qubits/sites. Since the width is known at compile time, the compiler can perform aggressive optimizations and autovectorization, resulting in performance improvements. `BasisString`, being a Rust const generic, cannot be used across the pyo3 boundary, and therefore its use in combination with `Bitset` allows for both performance and flexibility.

5.1.3 OperatorIndex

Although the `BasisString` data type is efficient for representing individual operators, it is not the most optimal way of representing many strings in a term sum. Note that each gate in a circuit will only affect a small number of qubits, and therefore for large systems, the majority of the bits in the bitmask will be zero. Additionally, propagation is commonly accompanied by weight truncation, which will prune strings with a large number of non-zero bits. Therefore, in order to store many operators $B_1, \dots, B_n$, we define the `OperatorIndex` data type which stores the indices of the non-trivial operations in a contiguous array for each key. This allows the propagator to easily identify which terms are affected by a gate. This architecture does not scale well if the number of non-trivial operations is large, and therefore `OperatorIndex` specifies an inline width W after which it will fall back to storing `BasisString<W>` in a hash map. In the presence of weight truncation, the inline width is fixed to twice the weight cutoff so that overflow into the hashmap never occurs. Additionally, the engine collects statistics on the number of terms that overflow into the hashmap, and automatically increases the inline width when necessary, with a hard cap of 32.

In addition to the `OperatorIndex` data type, we adopt the `InvertedIndex` view from MONOPROP [16], which provides a transposed view of the rows. That is, rather than delivering the indices of non-trivial operations for each term, it delivers the terms that contain a non-trivial operation for each index. This allows for the efficient computation of anticommutation relations.

For example, if the target gate’s generator acts on index $i, j, \dots$, then the `InvertedIndex` can be used to compute the combined parity of the terms across these indices. This flags the terms that anticommute with the generator, whose bitmasks are materialized and passed downstream for computing the new terms. Thus, we are able to avoid materializing the bitmasks of all terms, leading to performance improvements for large term sums.

5.2 Basis Algebra

The types described in the previous section store the structure of the operators, but do not provide any interpretation of the algebraic relations between them. The `Basis` trait provides a contract for the algebraic relations of the basis, i.e. it informs the engine how to interpret a `BasisString<W>` in the context of the basis. The `Basis` trait is implemented for both the Pauli and Majorana bases in `PauliAlgebra` and `MajoranaAlgebra`, respectively.

5.2.1 Pauli Strings

A Pauli string is a tensor product of Pauli operators for each qubit (21). It is often more convenient to represent a Pauli string in its symplectic representation, i.e. as a pair of bitmasks. For n qubits, we define length n bitmasks x and z such that

$$P = i^k \bigotimes_{i=1}^n X^{x_i} Z^{z_i} \quad (45)$$

Therefore, in combination with a 64-bit floating point number to represent the coefficient, a Pauli string can be stored as a struct of `(coeff, x, z)` in exactly $64 + 2n$ bits. Consistent with the design of term storage, Pauli strings have a frontend (`PauliString`) and backend (`BasisString<W> + PauliAlgebra`). For two Pauli strings $P_a = (x_a, z_a)$ and $P_b = (x_b, z_b)$ represented by their x and z bitmasks, we implement the product rule, commutation relations, operator weight, and overlap calculation with reference bitstrings, given as follows:

$$P_a P_b = i^k P_c \quad P_c := (x_a \oplus x_b, z_a \oplus z_b) \quad (46)$$

$$\{P_a, P_b\} = 0 \quad \text{iff} \quad p(x_a \cdot z_b) + p(z_a \cdot x_b) \mod 2 = 1 \quad (47)$$

$$\text{weight}(P_a) = p(x_a \vee z_a) \quad (48)$$

$$\langle \phi | P | \phi \rangle = \begin{cases} 0 & x \neq 0 \\ (-1)^{p(z \cdot \phi)} & x = 0 \end{cases} \quad (49)$$

where $p(\cdot)$ is the popcount function, $|\phi\rangle$ is a bitstring, $\oplus$ is the bitwise XOR operation, $\cdot$ is the bitwise AND operation, and $\vee$ is the bitwise OR operation. In addition to these algebraic relations, the data types also implement the necessary methods for serialization/deserialization, hashing, and equality comparison, enabling term I/O.

5.2.2 Majorana Monomials

The n -qubit Majorana operators (often called *Majorana monomials*), defined in (32) and repeated here for convenience, are given by

$$M = s \gamma_1^{b_1} \gamma_2^{b_2} \dots \gamma_{2n-1}^{b_{2n-1}} \gamma_{2n}^{b_{2n}} \quad (50)$$

where $s \in \mathbb{C}$ is a phase, b_i are binary exponents, and γ_i are the Majorana operators obeying the anticommutation relations.

$$\gamma_i^\dagger = \gamma_i, \quad \{\gamma_i, \gamma_j\} = 2\delta_{ij}I \quad (51)$$

Therefore, the Majorana operators can similarly be represented as a $2n$ -length bitmask, and the algebraic relations can be implemented as bitwise operations. Similar to Pauli strings, Majorana monomials have a frontend (`MajoranaMonomial`) and backend (`BasisString<W> + MajoranaAlgebra`), with the following algebraic relations implemented for two Majorana monomial bitmasks $M_a = (a)$ and $M_b = (b)$, with ℓ_a and ℓ_b active modes:

$$s = i^{h(\ell_a)} \quad h(\ell) := \begin{cases} 0 & \ell \equiv 0, 1 \pmod{4} \\ 1 & \ell \equiv 2, 3 \pmod{4} \end{cases} \quad (52)$$

$$M_a M_b = i^k M_c \quad M_c := (a \oplus b) \quad (53)$$

Here, k is an integer determined by reordering of anticommuting Majoranas and the phases s_a, s_b, s_c .

$$\{M_a, M_b\} = 0 \quad \text{iff} \quad \ell_a \ell_b + \ell_{a \cdot b} \equiv 1 \pmod{2} \quad (54)$$

$$\text{weight}(M_a) = \text{weight}(\text{JW}(M_a)) \quad (55)$$

$$\langle \phi | M_a | \phi \rangle = (-1)^{\lfloor n_{\text{paired}}/2 \rfloor} \prod_{k: \text{paired \& active}} (2\phi_k - 1) \quad (56)$$

where $\text{JW}(\cdot)$ is the Jordan-Wigner transformation, $|\phi\rangle$ is a bitstring/Slater determinant, and

s is the phase required to ensure the Hermiticity of the Majorana monomial. Note that this assumes that $a_{2k} = a_{2k-1}$ for all k , otherwise $\langle \phi | M_a | \phi \rangle = 0$. The weight is taken to be the *Pauli weight* of the Jordan-Wigner-transformed monomial, in order to ensure consistency with noise models. The serialization/deserialization, hashing, and equality comparison methods are also implemented for Majorana monomials.

5.3 Coefficient Representation

The propagation engines are designed to be generic over the coefficient representation, allowing the same engine to be used for numeric propagation with different floating point precisions, as well as symbolic propagation. This functionality is provided by the `CoeffRepr` trait, which mainly wraps the coefficient type and provides a contract for the necessary algebraic operations. The numeric propagators currently support double (`f64`) and single (`f32`) precision floating point numbers.

The symbolic coefficients (`SymbolicCoeff`) are represented as directed acyclic graphs (DAGs) of algebraic operations, as is common in computer algebra systems (CAS). The arithmetic operations for scaling by a numerical coefficient, adding two coefficients, and multiplying by a symbolic $\cos(\theta)$ or $\sin(\theta)$ term are all represented as nodes in the DAG. The nodes are wrapped in atomic reference-counted pointers (`Arc`) to allow for sharing of subexpressions across threads. Therefore, the symbolic propagation becomes a DAG construction, and the compilation to the final surrogate model walks the DAG to generate the final expression. The `Simplify` object identifies and merges shared symbolic histories. As one might expect, this is significantly more restricted than merging in a numerical propagator, and consequently symbolic propagation is more memory intensive and slower than numerical propagation.

5.4 Propagation Engine

The propagation engine is responsible for applying gates, noise channels, and truncation policies to the operator in order to carry out the propagation. The observable is first divided over n partitions, one for each thread. This enables multithreading via `rayon`. Additionally, the partitions, which disjointly divide the term space, are pinned to their corresponding spawner threads on Linux

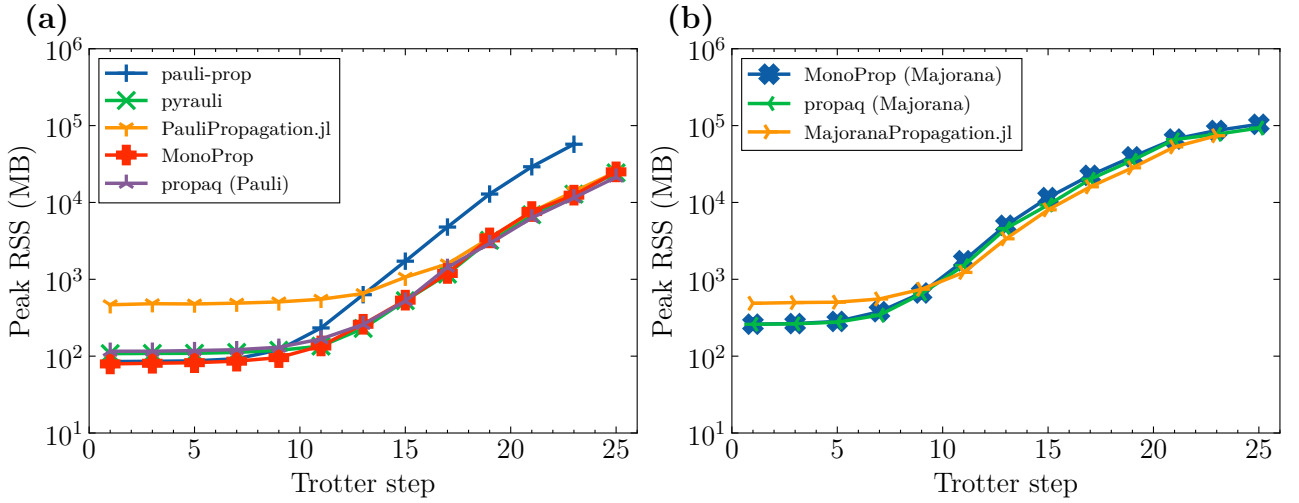


Figure 10: Peak memory usage across various backends for computing expectation values of (a) Single Z observable trotterized time evolution of a 2D TFIM on a 6×6 square lattice ($J = 1.0$, $h = 0.5$, $\Delta t = 0.1$) (b) Single Z observable trotterized time evolution of a 2D Hubbard model on a 3×3 square lattice ($t = 1.0$, $U = 4.0$, $\Delta t = 0.1$). All backends use a coefficient cutoff of 10^{-6} and 64 threads, except for the single-threaded pauli-prop package.

systems to promote cache locality of the term sums. The circuit is applied in reverse order, and as a result the noise channels are applied prior to each layer.

For each gate, the propagator first checks if it can be deferred through the Clifford deferral mechanism (Sec 2.7). If so, its adjoint action is stored in a `CliffordTableau`. Once a non-Clifford gate is encountered later, it is conjugated through the tableau and applied to the term sum. Next, each owner thread identifies the terms affected by the gate through the `OperatorIndex/InvertedIndex` data structures described in Sec. 5.1.

From here, the affected terms' bitmasks in each partition are materialized by the owner thread. In the presence of coefficient truncation, the propagator estimates the coefficient magnitude of new terms and might choose not to produce them if they do not meet the threshold. For the new terms that are produced, they are then pruned according to given truncation policies. Terms not meeting the coefficient cutoff are stashed rather than being immediately dropped. It is common for an owner thread to produce terms not in its partition, and these terms must be routed to the correct owner thread. This is batched and done after each gate application across all threads to reduce the overhead of thread communication.

In addition to this mechanism, if any term that was stashed due to truncation is later found in a partition, it is retrieved and added back to the term sum. Since the term is already being stored

in memory, this rescue mechanism minimizes information loss while being free in memory. Note that terms dropped due to weight truncation are not rescued, since this truncation is structural. The scan over the terms during this phase is used to compute truncation statistics, which are logged to file if logging is enabled. After the routing, the noise channels are applied, and the next gate is processed. This continues until all gates have been applied, at which point either the final term sum is returned, or the expectation value is calculated by computing term overlaps with the initial state.

6 Memory and Thread Performance

As shown in Fig. 1, Propaq meets or exceeds state-of-the-art Heisenberg propagation packages in terms of runtime. Here we show the analogous benchmark for memory. Specifically, we compare memory usage for the same benchmark in Fig. 1, namely computing a Pauli Z expectation value for two-dimensional 6×6 transverse-field Ising model and an 3×3 Hubbard model. These results are shown in Fig. 10. In these benchmarks, all simulators use the same coefficient cutoff of 10^{-6} . As can be seen, despite the differences in runtime between packages in Fig. 1, they are much more similar in terms of memory usage. In this case, we still see Propaq performing on par or better than current state of the art, both in

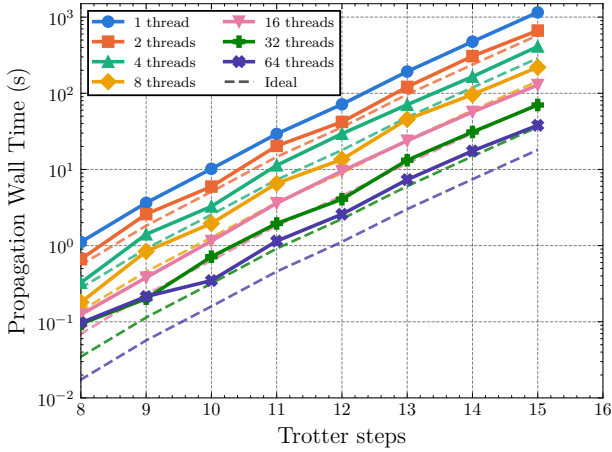


Figure 11: Propagation wall-clock time vs. Trotter step count for Propaq running on 1–64 threads on a 2D 6×6 TFIM, compared to ideal linear scaling.

the Pauli and the Majorana basis. We expect to see the arbitrary basis support in Propaq enable much more efficient simulations for certain applications, both in terms of memory and runtime, and seek to explore these applications in future work.

Additionally, we study how performance scales with the number of threads. To do so, we record the wall-clock time to compute the energy of a two-dimensional 6×6 transverse-field Ising model vs the number of Trotter steps. These results, shown in Fig. 11, demonstrate that multiple threads can result in significant speedups. Indeed, we roughly see a 10-20x speedup from one to 64 threads in this example. Additional threads on a single CPU, or on multiple CPUs, can further improve performance.

7 Conclusion

We introduced Propaq, a highly general and flexible package for Heisenberg propagation. We showed that Propaq performs as good or better than state-of-the-art Heisenberg simulation packages in terms of both runtime and memory. Because Propaq contains multiple features that stand out from current software, we expect it to be useful for both quantum software developers and researchers. Notably, several of these features open the possibility for tackling open research questions, for example choosing the optimal basis for Heisenberg propagation and choosing the best circuit partitioning for hybrid Schrödinger-Heisenberg simulation. We seek to tackle these

open questions in future work enabled by Propaq, and make it freely and openly available to the community for use.

Acknowledgments

We acknowledge the use of generative AI for assistance writing some parts of the code in Propaq, and for creating Fig. 3. We acknowledge [38] for making plots.

References

- [1] Ryan LaRose. A brief history of quantum vs classical computational advantage. *Quantum*, 10:2198, 2026. arXiv:2412.14703 [quant-ph].
- [2] Daniel Gottesman. The Heisenberg Representation of Quantum Computers, July 1998.
- [3] Guifré Vidal. Efficient Classical Simulation of Slightly Entangled Quantum Computations. *Physical Review Letters*, 91(14):147902, October 2003.
- [4] Xun Gao and Luming Duan. Efficient classical simulation of noisy quantum computation, October 2018. arXiv:1810.03176 [quant-ph].
- [5] Dorit Aharonov, Xun Gao, Zeph Landau, Yunchao Liu, and Umesh Vazirani. A polynomial-time classical algorithm for noisy random circuit sampling. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, STOC 2023, page 945–957, New York, NY, USA, 2023. Association for Computing Machinery.
- [6] Thomas Schuster, Chao Yin, Xun Gao, and Norman Y. Yao. A polynomial-time classical algorithm for noisy quantum circuits, October 2024.
- [7] Jon Nelson, Joel Rajakumar, Dominik Hangleiter, and Michael J. Gullans. Polynomial-time classical simulation of noisy quantum circuits with naturally fault-tolerant gates. (arXiv:2411.02535), January 2026. arXiv:2411.02535 [quant-ph].
- [8] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. Evidence for the utility of

- quantum computing before fault tolerance. *Nature*, 618(7965):500–505, June 2023.
- [9] Tomislav Begušić, Johnnie Gray, and Garnet Kin-Lic Chan. Fast and converged classical simulations of evidence for the utility of quantum computing before fault tolerance. *Science Advances*, 10(3):eadk4321, January 2024.
 - [10] Tomislav Begušić and Garnet Kin-Lic Chan. Real-time operator evolution in two and three dimensions via sparse pauli dynamics. *PRX Quantum*, 6(2):020302, April 2025. arXiv:2409.03097 [quant-ph].
 - [11] Manuel S. Rudolph, Tyson Jones, Yanting Teng, Armando Angrisani, and Zoë Holmes. Pauli Propagation: A Computational Framework for Simulating Quantum Systems, May 2025.
 - [12] Aaron Miller, Joachim Favre, Zoë Holmes, Özlem Salehi, Rahul Chakraborty, Anton Nykänen, Zoltán Zimborás, Adam Glos, and Guillermo García-Pérez. Simulation of Fermionic circuits using Majorana Propagation, December 2025.
 - [13] Matteo D’Anna, Jannes Nys, and Juan Carrasquilla. Majorana string simulation of nonequilibrium dynamics in two-dimensional lattice fermion systems. <https://arxiv.org/abs/2511.02809>, 2025.
 - [14] Qiskit addons team. Pauli prop. <https://github.com/Qiskit/pauli-prop>, 2025.
 - [15] zeFresk. pyrauli GitHub. <https://github.com/zeFresk/pyrauli>, August 2026.
 - [16] Simulation of fermionic circuits using majorana propagation. <https://github.com/algorithmiq/monoprop>, March 2025.
 - [17] Bryce Fuller, Minh C. Tran, Danylo Lykov, Caleb Johnson, Max Rossmannek, Ken Xuan Wei, Andre He, Youngseok Kim, DinhDuy Vu, Kunal Sharma, Yuri Alexeev, Abhinav Kandala, and Antonio Mez-zacapo. Improved Quantum Computation using Operator Backpropagation, February 2025. arXiv:2502.01897 [quant-ph].
 - [18] Anna Krasznai, Sergei Rutkevich, and Gábor Takács. Confinement in the three-state Potts quantum spin chain in extreme ferromagnetic limit, April 2026.
 - [19] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, July 2021.
 - [20] Mario Motta, Kevin J. Sung, K. Birgitta Whaley, Martin Head-Gordon, and James Shee. Bridging physical intuition and hardware efficiency for correlated electronic states: The local unitary cluster Jastrow ansatz for electronic structure. *Chemical Science*, 14(40):11213–11227, 2023.
 - [21] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18):180509, November 2017.
 - [22] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J. Zeng. Digital zero noise extrapolation for quantum error mitigation. *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 306–316, October 2020. arXiv: 2005.10921.
 - [23] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O’Brien. A variational eigenvalue solver on a quantum processor. *Nature Communications*, 5(1), December 2014. arXiv: 1304.3061.
 - [24] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A Quantum Approximate Optimization Algorithm, November 2014. arXiv:1411.4028.
 - [25] Sumeet Khatri, Ryan LaRose, Alexander Poremba, Lukasz Cincio, Andrew T. Sornborger, and Patrick J. Coles. Quantum-assisted quantum compiling. *Quantum*, 3:140, May 2019.
 - [26] Ryan LaRose, Arkin Tikku, Étude O’Neel-Judy, Lukasz Cincio, and Patrick J. Coles. Variational quantum state diagonalization. *npj Quantum Information*, 5(1):1–10, June 2019.
 - [27] Carlos Bravo-Prieto, Ryan LaRose, M. Cerezo, Yigit Subasi, Lukasz Cincio, and Patrick J. Coles. Variational Quantum Linear Solver. *Quantum*, 7:1188, November 2023.
 - [28] M. Cerezo, Andrew Arrasmith, Ryan Babush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, September 2021.
 - [29] Scott Aaronson and Daniel Gottesman. Im-

- proved Simulation of Stabilizer Circuits. *Physical Review A*, 70(5):052328, November 2004.
- [30] Manuel S. Rudolph, Tyson Jones, Yanting Teng, Armando Angrisani, and Zoë Holmes. Pauli Propagation: A Computational Framework for Simulating Quantum Systems, May 2026.
 - [31] Ryan LaRose. Overview and Comparison of Gate Level Quantum Software Platforms. *Quantum*, 3:130, March 2019.
 - [32] Kevin J. Sung, Inho Choi, Mirko Amico, Bartholomew Andrews, Esra Ayantuna, Yukio Kawashima, Wan-Hsuan Lin, David Omanovic, Samuele Piccinelli, Javier Robledo Moreno, Abdullah Ash Saki, James Shee, Soyoung Shin, Minh C. Tran, Kento Ueda, Haimeng Zhang, and Mario Motta. ff-sim: Faster simulation of fermionic quantum circuits, May 2026. arXiv:2605.03123 [quant-ph].
 - [33] Jarrod R. McClean, Ian D. Kivlichan, Kevin J. Sung, Damian S. Steiger, Yudong Cao, Chengyu Dai, E. Schuyler Fried, Craig Gidney, Brendan Gimby, Thomas Häner, Tarini Hardikar, Vojtěch Havlíček, Cupjin Huang, Zhang Jiang, Matthew Neeley, Thomas O’Brien, Isil Ozfidan, Maxwell D. Radin, Jhonathan Romero, Nicholas Rubin, Nicolas P. D. Sawaya, Kanav Setia, Sukin Sim, Mark Steudtner, Wei Sun, Fang Zhang, and Ryan Babbush. OpenFermion: The Electronic Structure Package for Quantum Computers. *arXiv:1710.07629 [physics, physics:quant-ph]*, October 2017. arXiv: 1710.07629.
 - [34] Johnnie Gray. quimb: A python package for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, September 2018.
 - [35] Hrishikesh Belagali and Ryan LaRose. Propaq GitHub. <https://github.com/hkbelagali/propaq>, September 2026. original-date: 2026-05-20T23:03:52Z.
 - [36] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter J. Karalekas, Andre A. Alves, Piotr Czarnik, Mohamed El Mandouh, Max H. Gordon, Yousef Hindy, Aaron Robertson, Purva Thakre, Misty Wahl, Danny Samuel, Rahul Mistri, Maxime Tremblay, Nick Gardner, Nathaniel T. Stemen, Nathan Shammah, and William J. Zeng. Mitiq: A software package for error mitigation on noisy quantum computers. *Quantum*, 6:774, August 2022. arXiv:2009.04417 [quant-ph].
 - [37] Cristina Cirstoiu, Silas Dilkes, Daniel Mills, Seyon Sivarajah, and Ross Duncan. Volumetric Benchmarking of Error Mitigation with Qermit. *Quantum*, 7:1059, July 2023. arXiv:2204.09725 [quant-ph].
 - [38] John D. Garrett. SciencePlots GitHub. <https://doi.org/10.5281/zenodo.4106649>, September 2021.

A Weyl-Heisenberg propagation

In this section, we describe Heisenberg propagation in the Weyl-Heisenberg basis, which is a generalization of the Pauli basis to qudits. We provide the algebraic relations for the Weyl-Heisenberg operators and code snippets subclassing Propaq's classes to implement Weyl-Heisenberg propagation.

A.1 Weyl-Heisenberg Algebra

On one *qudit* of dimension d , the Weyl-Heisenberg operators are defined by the following relations:

$$X|j\rangle := |j+1 \bmod d\rangle, \quad Z|j\rangle := \omega^j|j\rangle, \quad \omega := e^{2\pi i/d} \quad (57)$$

obeying $X^d = Z^d = I$ and $ZX = \omega XZ$. Then, a Weyl-Heisenberg string on n qudits can be represented using two exponent vectors $a, b \in \mathbb{Z}_d^n$ as

$$W(a, b) := \tau^{a \cdot b} X^a Z^b, \quad X^a Z^b := \bigotimes_k X^{a_k} Z^{b_k} \quad (58)$$

The prefactor τ is a phase factor, for which we will fix the convention

$$\tau := \begin{cases} \omega^{(d+1)/2} & d \text{ odd} \\ e^{i\pi/d} & d \text{ even} \end{cases} \quad (59)$$

Note that for $d = 2$, these relations reduce to the Pauli algebra. Computing the product of two Weyl-Heisenberg strings $W(u)$ and $W(v)$ yields

$$W(v)W(u) = \tau^{E(v,u)} W(\bar{s}, \bar{t}), \quad \begin{aligned} \bar{s} &= v_a + u_a \bmod d \\ \bar{t} &= v_b + u_b \bmod d \end{aligned} \quad (60)$$

where the exponent $E(v, u)$ is given by

$$E(v, u) = v_a \cdot v_b + u_a \cdot u_b + 2u_a \cdot v_b - \bar{s} \cdot \bar{t} \pmod{2d} \quad (61)$$

This means that the commutation relations of Weyl-Heisenberg strings are as follows:

$$W(u)W(v) = \omega^{\sigma(v,u)} W(v)W(u) \quad \sigma(v, u) = v_a \cdot u_b - u_a \cdot v_b \pmod{d} \quad (62)$$

where σ is the symplectic form over $\mathbb{Z}_d$. Finally, we will need the overlap of a Weyl-Heisenberg string with a bitstring state $|f\rangle = |f_1, f_2, \dots, f_n\rangle$, which is given by

$$\langle f | W(a, b) | f \rangle = \delta_{a,0} \omega^{b \cdot f} \quad (63)$$

We now have all the necessary relations to set up the term representation for Weyl-Heisenberg strings.

Listing A.1. Subclassing Propaq to implement Weyl-Heisenberg strings.

```
1 from propaq.datatypes import AbstractTerm
2
3 def tau_pow(d: int, e: int) -> complex:
4     """
5     Compute the phase factor  $\tau^e$ 
6     """
7     # Determine k based on the parity of d
8     k = 1 if d % 2 == 0 else d+1
9     return cmath.exp(1j * math.pi * k * (e % (2*d)) / d)
10
11 def omega_pow(d: int, e: int) -> complex:
```

Listing A.1. (continued)

```

12     """
13     Compute the phase factor  $\omega^e$ 
14     """
15     return cmath.exp(2j * math.pi * (e % d) / d)
16
17 @dataclass(frozen=True)
18 class WeylString(AbstractTerm):
19     """ $W(a, b) = \tau^{a \cdot b} X^a Z^b$ """
20
21     a: tuple[int, ...] # exponent vector for X,  $a \in \mathbb{Z}_d^n$ 
22     b: tuple[int, ...] # exponent vector for Z,  $b \in \mathbb{Z}_d^n$ 
23     d: int
24
25     @property
26     def n_units(self) -> int:
27         return len(self.a)
28
29     @property
30     def weight(self) -> int:
31         return sum(1 for ai, bi in zip(self.a, self.b) if ai or bi)
32
33     def __matmul__(self, other: "WeylString") -> tuple[complex, "WeylString"]:
34         d, e, na, nb = self.d, 0, [], []
35         for va, vb, ua, ub in zip(self.a, self.b, other.a, other.b):
36             s, t = (va + ua) % d, (vb + ub) % d
37             e += va * vb + ua * ub + 2 * ua * vb - s * t # (Eq 61)
38             na.append(s)
39             nb.append(t)
40         return tau_pow(d, e), WeylString(tuple(na), tuple(nb), d) # (Eq 60)
41
42     def dagger(self) -> tuple[complex, "WeylString"]:
43         inv = WeylString(
44             tuple((-x) % self.d for x in self.a), tuple((-x) % self.d for x in self.b), self.d
45         )
46         phase, _ = self @ inv
47         return 1 / phase, inv
48
49     def symplectic(self, other: "WeylString") -> int:
50         return sum(
51             va * ub - ua * vb for va, vb, ua, ub in zip(self.a, self.b, other.a, other.b)
52         ) % self.d # (Eq 62)
53
54     def commutes_with(self, other: "WeylString") -> bool:
55         return self.symplectic(other) == 0
56
57     def trace_with_diag_state(self, diag_state: int | tuple[int, ...]) -> complex:
58         if any(self.a):
59             return 0j
60         if isinstance(diag_state, int):
61             diag_state = tuple((diag_state // self.d * k) % self.d for k in range(self.n_units))
62         return omega_pow(self.d, sum(bi * fi for bi, fi in zip(self.b, diag_state))) # (Eq 63)
63
64     def to_bytes(self) -> bytes:
65         return bytes(self.a) + bytes(self.b) + bytes([self.d])
66
67     @classmethod
68     def from_bytes(cls, data: bytes, n_units: int) -> "WeylString":
69         a, b, d = data[:n_units], data[n_units : 2 * n_units], data[-1]
70         return cls(tuple(a), tuple(b), d)
71
72     def __hash__(self) -> int:
73         return hash((self.a, self.b))
74
75     def __eq__(self, other: object) -> bool:
76         return (
77             isinstance(other, WeylString) and self.a == other.a and self.b == other.b
78         )
79
80
81 def identity(n: int, d: int) -> WeylString:

```

Listing A.1. (continued)

```

82     return WeylString((0,) * n, (0,) * n, d)
83
84
85 def single(n: int, d: int, k: int, ai: int, bi: int) -> WeylString:
86     """ $X^{a_i} Z^{b_i}$  acting on site  $k$  alone."""
87     a, b = [0] * n, [0] * n
88     a[k], b[k] = ai % d, bi % d
89     return WeylString(tuple(a), tuple(b), d)

```

A.2 Weyl-Heisenberg Gates

A Weyl-Heisenberg string is not generally Hermitian for $d > 2$. We'll consider only the Hermitian component of the Weyl-Heisenberg strings as unitary gate generators:

$$H_g = \frac{1}{2}(W_g + W_g^\dagger) \quad U_g(\theta) = \exp\left(-i\frac{\theta}{2}H_g\right) \quad (64)$$

Note that for $d = 2$, we have $W_g = W_g^\dagger$ and therefore $U = \exp(-i\theta W_g)$, which is the usual Pauli gate. Because $W_g^d = I$, we can write the eigenprojectors of W_g as $\Pi_k = \frac{1}{d} \sum_m \omega^{-km} W_g^m$ for $k = 0, \dots, d-1$. Then, we can write the unitary gate as a linear combination of the eigenprojectors, where the coefficients are given by:

$$U_g(\theta) = \sum_{m=0}^{d-1} c_m W_g^m, \quad c_m = \frac{1}{d} \sum_{k=0}^{d-1} \exp\left\{-i\frac{\theta}{2} \cos(2\pi k/d)\right\} \omega^{-km} \quad (65)$$

Once again, note that for $d = 2$ we recover the familiar $c_0 = \cos(\theta/2)$ and $c_1 = -i \sin(\theta/2)$. Let $s = \sigma(v, g)$, so $W_g^j W(v) = \omega^{js} W(v) W_g^j \forall j$ as in Eq. (62). We are now equipped to derive the propagation rule for a Weyl-Heisenberg gate's action:

$$U_g^\dagger W(v) U_g = \sum_{m, m'} \bar{c}_{m'} c_m W_g^{-m'} W(v) W_g^m = \sum_{m, m'} \bar{c}_{m'} c_m \omega^{-m's} W(v) W_g^{m-m'} \quad (66)$$

and allowing $j = m - m'$, we have

$$U_g(\theta)^\dagger W(v) U_g(\theta) = \sum_{j=0}^{d-1} \Lambda_s[j] W(v) W_g^j \quad \Lambda_s[j] = \sum_{m=0}^{d-1} \bar{c}_m c_{m+j} \omega^{-ms} \quad (67)$$

Therefore, the propagation of a Weyl-Heisenberg string through a gate can result in up to d branches. Λ is a $d \times d$ table dependent on the gate, and each term in the sum contributes its symplectic form s to the phase factor.

Consider the equation for $d = 2$. Then, we have $c_0 = \cos(\theta/2)$, and $c_1 = -i \sin(\theta/2)$. Since we operate over $\mathbb{Z}_2$, we have $s = 0$, or $s = 1$. When $s = 0$, we have

$$\Lambda_0[0] = \bar{c}_0 c_0 + \bar{c}_1 c_1 = \cos^2(\theta/2) + \sin^2(\theta/2) = 1 \quad (68)$$

$$\Lambda_0[1] = \bar{c}_0 c_1 + \bar{c}_1 c_0 = -i \cos(\theta/2) \sin(\theta/2) + i \sin(\theta/2) \cos(\theta/2) = 0 \quad (69)$$

When $s = 1$, we have

$$\Lambda_1[0] = \cos^2(\theta/2) - \sin^2(\theta/2) = \cos(\theta) \quad (70)$$

$$\Lambda_1[1] = -i \cos(\theta/2) \sin(\theta/2) - i \sin(\theta/2) \cos(\theta/2) = -i \sin(\theta) \quad (71)$$

This corresponds exactly to the Pauli propagation rule. We can now implement the Weyl-Heisenberg generator parameterization:

Listing A.2. Weyl-Heisenberg generator parameterization.

```

1  @lru_cache(maxsize=None)
2  def _gate_coefficients(d: int, theta: float) -> tuple[complex, ...]:
3      """ $c_m$  such that  $U_g(\theta) = \sum_m c_m W_g^m$ ."""
4      k = np.arange(d)
5      eigs = np.exp(-0.5j * theta * np.cos(2 * np.pi * k / d))
6      return tuple(np.sum(eigs * np.exp(-2j * np.pi * k * m / d)) / d for m in range(d)) # (Eq 65)
7
8
9  class WeylRotation:
10     """
11     A unitary gate generated by the Hermitian component of a Weyl string.
12     """
13
14     def __init__(self, generator: WeylString, angle: float):
15         d, n = generator.d, generator.n_units
16         self.generator, self.angle = generator, angle
17
18         powers = [(1 + 0j, identity(n, d))]
19         for _ in range(d - 1):
20             phase, key = powers[-1]
21             step_phase, step_key = key @ generator
22             powers.append((phase * step_phase, step_key))
23         phase, key = powers[-1]
24         step_phase, step_key = key @ generator
25         if step_key != identity(n, d) or abs(phase * step_phase - 1) > 1e-12:
26             raise ValueError("generator does not satisfy  $W^d = 1$ ")
27         self.powers = powers
28
29         c = np.asarray(_gate_coefficients(d, angle))
30         omega = np.array([[omega_pow(d, -m * s) for m in range(d)] for s in range(d)])
31
32         #  $\Lambda_s[j]$ , (Eq 67)
33         self.table = np.array(
34             [[np.sum(np.conj(c) * np.roll(c, -j) * omega[s]) for j in range(d)] for s in range(d)]
35         )
36
37     def conjugate(key: WeylString, coeff: complex, rot: WeylRotation):
38         """Yield the (child, coefficient) pairs of one term under one gate."""
39         row = rot.table[key.symplectic(rot.generator)]
40         for j in range(key.d):
41             weight = row[j]
42             if weight == 0:
43                 continue
44             power_phase, power_key = rot.powers[j]
45             phase, child = key @ power_key
46             yield child, coeff * weight * power_phase * phase

```

Now, we have everything required to implement the Weyl-Heisenberg propagator. We will subclass `AbstractPropagator` and write small wrappers around the truncation and noise policies to make them compatible with the Weyl-Heisenberg term representation.

Listing A.3. Subclassing Propaq to implement Weyl-Heisenberg propagation.

```

1  from propaq.propagators import AbstractPropagator
2  from propaq.datatypes import DictTermSum
3
4  class WeylTermSum(DictTermSum[WeylString]):
5      term_type = WeylString # enables file I/O
6
7  class WeylPropagator(AbstractPropagator):
8      """Heisenberg propagation of Weyl-Heisenberg observables through qudit circuits."""
9
10     term_sum_type = WeylTermSum
11
12     def apply_gate(self, term: WeylString, coeff: complex, rotation: "WeylRotation"):
13         """Yield the (child, coefficient) pairs of one term under one gate."""
14         return conjugate(term, coeff, rotation)

```

We have provided an example implementation of this Weyl-Heisenberg propagator on Propaq's

website, extensively validating the implementation against density matrix propagation and validating against known analytical results for the Potts model. We also demonstrate the out-of-box use of zero noise extrapolation. This can be found at https://hkbelagali.github.io/propaq/examples/usage/11_custom_basis_qudit_weyl/.

B Custom noise plugin implementation in C

Listing B.1. Implementing uniform depolarizing noise in C for Propaq.

```

1 // uniform_noise.c
2 #include <math.h>
3 #include <stdint.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 #define PROPAQ_NOISE_ABI_VERSION 1u // ABI version 1.0
8
9 typedef struct {
10     double damping;
11 } Ctx;
12
13 uint32_t propaq_noise_abi_version(void) {
14     return PROPAQ_NOISE_ABI_VERSION; // Return the ABI version of the plugin for compatibility checks
15 }
16
17 // Parse a JSON string to extract the model parameters
18 static double parse_damping(const char* config_json) {
19     if (config_json == NULL) return 0.0; // Default damping if no config is provided
20     const char* key = strstr(config_json, "\"damping\"");
21
22     if (key == NULL) return 0.0; // Default damping if key not found
23
24     const char* colon = strchr(key, ':');
25     if (colon == NULL) return 0.0;
26     return atof(colon + 1);
27 }
28
29 // Create a new noise model context
30 void* propaq_noise_create(const char* config_json) {
31     Ctx* ctx = (Ctx*)malloc(sizeof(Ctx));
32     ctx->damping = parse_damping(config_json);
33     return ctx;
34 }
35
36 // Free alloc for model
37 void propaq_noise_destroy(void* ctx) { free(ctx); }
38
39 // Implement the noise application
40 // Args:
41 //   ctx: pointer to the noise model context
42 //   basis_kind: the enum for the basis (Pauli, Majorana, etc.)
43 //   words: pointer to the array of strings
44 //   n_words: number of words in the array
45 //   n_units: number of units (qubits, fermions, etc.)
46 //   weight: the weight of the operator
47 //   layer_index: the index of the current layer in the circuit
48 //   n_layers: the total number of layers in the circuit
49 double propaq_noise_factor(void* ctx, uint32_t basis_kind, const uint64_t* words, size_t n_words,
50     uint32_t n_units, uint32_t weight, uint32_t layer_index,
51     uint32_t n_layers) {
52     (void)basis_kind; (void)words; (void)n_words; (void)n_units;
53     (void)layer_index; (void)n_layers;
54     double damping = ((Ctx*)ctx)->damping;
55     return exp(-damping * (double)weight); // exp(damping * w[P])
56 }

```